

JavaScript, Types and Variables

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

JavaScript in One Minute

- Programming language of the web, created in **1995** by Brendan Eich
- **Not** related to Java; the name was a marketing choice
- Runs almost everywhere: browsers, servers, phones, tiny devices
- If you know variables, functions, and loops, much will look familiar
- This course teaches its **current, modern form**

ECMAScript: The Standard

- **ECMAScript (ES)** is the written *specification*, the rulebook
- **JavaScript** is the language you actually *write*, an implementation
- Maintained by the **TC39** committee, which ships one edition per year
- **ES2015 (ES6)** was the big modernizing release, the baseline here
- **ES2017:** `async` / `await` · **ES2020:** optional chaining `?.`, nullish `??`
- **ES2022:** `Object.hasOwn`, top-level `await`, class fields
- **ES2025:** iterator helpers, new `Set` methods, `RegExp.escape`
- **ES2026 (current):** `Error.isError`, `Math.sumPrecise`, `Array.fromAsync`
- Everyday usage: say "JavaScript" for the language, "ES2020" for a version

Where JavaScript Runs

- **In the browser:** change the page (DOM), respond to events, `fetch` data
- Browser-only globals: `window`, `document`, `fetch`
- **On the server with Node.js:** the same language, outside the browser
- Node extras: file system, network, but **no** `document` or `window`
- An **engine** actually runs your code, and comes bundled, never installed alone
- **V8** (Chrome, Edge, Node) · **SpiderMonkey** (Firefox) · **JavaScriptCore** (Safari)

ES2015 Modernized the Language

```
let count = 0;           // block-scoped variables
const name = "Ada";      // constants
const greet = () => {};   // arrow functions
const msg = `Hi ${name}`; // template literals
const [a, b] = [1, 2];    // destructuring
```

- Baseline for this course is **ES2015+**
- Older code you meet in the wild will look different; we will note where

Dynamic and Loosely Typed

- **Dynamic**: types are checked at **runtime**, not ahead of time
- **Loosely typed**: JS often **converts** types for you (coercion)
- The **value** has a type, not the variable

```
let x = 10;    // x is a number
x = "hello";   // now x is a string
x = false;    // now x is a boolean
```

Primitive Types vs Objects

- Primitives: `number`, `string`, `boolean`, `undefined`, `null`, `symbol`, `bigint`
- Everything else is an **object** (arrays, functions, plain objects)

```
let age = 30;           // number (primitive)
let name = "Alice";     // string (primitive)
let nothing = null;     // null (primitive)
let notSet;             // undefined (primitive)
let user = { name: "Alice" }; // object
let numbers = [1, 2, 3]; // object (array)
```

The `typeof` Operator

```
typeof 42;           // "number"
typeof "hello";      // "string"
typeof true;         // "boolean"
typeof undefined;    // "undefined"
typeof { a: 1 };     // "object"
typeof [1, 2, 3];     // "object" (arrays are objects)
typeof function () {}; // "function"
typeof null;         // "object" (a long-standing bug)
```

- For `null`, check `value === null` instead of using `typeof`

Declaring: `let`, `const`, `var`

- `let`: for values that can **change**
- `const`: for values that should **not be reassigned**
- `var`: the old way; tricky scope, avoid in new code

```
let score = 0;  
score = 10;      // OK  
  
const pi = 3.14;  
// pi = 3.14159; // Error: Assignment to constant variable
```

- A `const` **must** be given a value when declared

`const` Protects the Binding, Not the Value

```
const settings = { theme: "dark" };

settings.theme = "light"; // OK - the object changed
// settings = {};        // Error - the binding cannot be reassigned

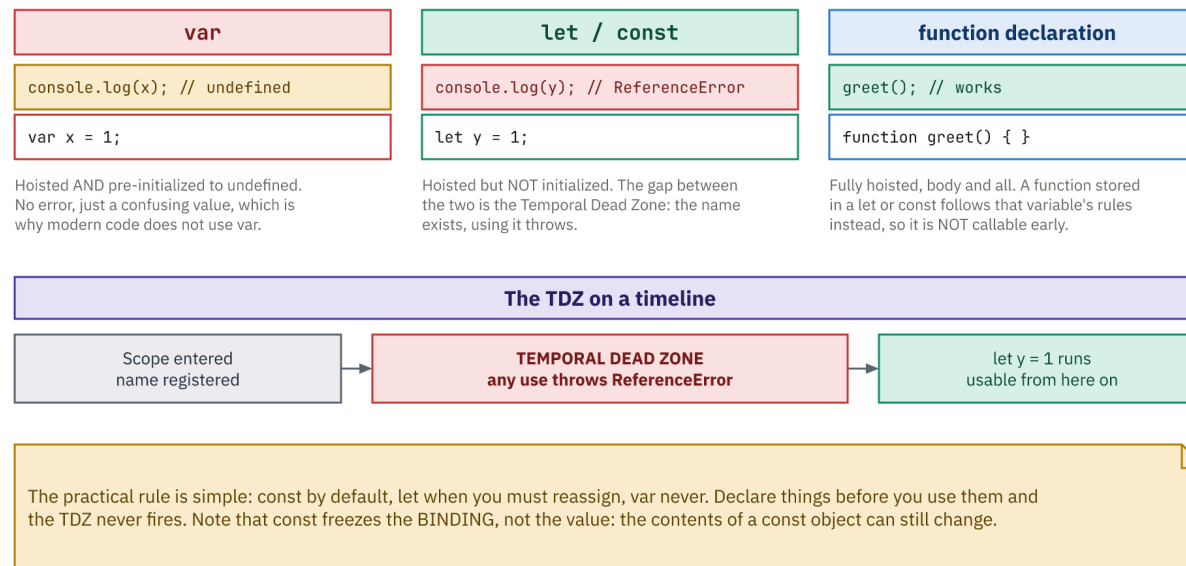
const list = [1, 2];
list.push(3);             // OK - [1, 2, 3]
```

- `const` means "this name always points at this value"
- It does **not** freeze the object; use `Object.freeze` for that

Declaring vs Initializing, and Naming

Hoisting and the Temporal Dead Zone

Declarations are registered before the code runs. What differs is whether you may touch them yet.



- Declared but not yet initialized: `let` and `const` are in the **TDZ**
- Names start with a letter, `_` or `$`; never a reserved word

Scope: Where a Variable Exists

- A **block** is anything inside `{ }` (as in `if` or `for`)
- `let` / `const` are **block-scoped**: they exist only inside the block

```
if (true) {  
  let message = "Hello";  
  console.log(message); // "Hello"  
}  
console.log(message); // Error: message is not defined
```


`var` Is Function-Scoped

```
if (true) {  
  var test = "I escape the block";  
}  
console.log(test); // "I escape the block"  
  
if (true) {  
  let test2 = "Block only";  
}  
console.log(test2); // Error: test2 is not defined
```

- `var` is function-scoped, hoisted, and confusing; prefer `let` / `const`

Assignment Evaluates the Right Side First

```
let a = 2;  
let b = 1 + 2; // evaluate 1 + 2, then store 3  
  
let c = a;      // a evaluates to 2, so 2 is stored in c  
let d = a + 2;  // 4  
  
a = 5;          // reassign a  
  
console.log(c); // 2 - c copied the VALUE, not the variable  
console.log(a); // 5
```

Value Types vs Reference Types

Value vs Reference

Primitives copy the value. Objects copy the reference: two names, one object.

Primitives copy the VALUE

```
let a = 10;  
let b = a; // copies the number  
b = 20;  
// a is still 10
```

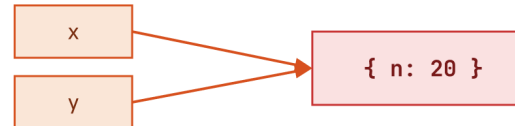
a → 10

b → 20

Two separate boxes. Changing one cannot touch the other. Same for string, boolean, number, null, undefined, symbol, bigint.

Objects copy the REFERENCE

```
const x = { n: 10 };  
const y = x; // copies the arrow, not the box  
y.n = 20;  
// x.n is now 20 too
```



One object, two names. Arrays and functions are objects too, so they behave the same way.

To get a real copy, spread it: `const z = { ...x }` or `const arr2 = [ ...arr ]`
But spread is SHALLOW: a nested object is still shared. Use `structuredClone(x)` for a deep copy.
Also why `const` does not mean immutable: it fixes the arrow, not the box it points at.

- Primitives copy the **value**; objects copy the **reference**
- Two names, one object; changing either changes both

Operators and Strings

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Arithmetic Operators

```
5 + 3;    // 8    addition
5 - 3;    // 2    subtraction
5 * 3;    // 15   multiplication
5 / 3;    // 1.666... division - always a float, never integer
5 % 3;    // 2    remainder ("modulo")
5 ** 3;   // 125  exponentiation
```

- `/` never truncates: `10 / 4` is `2.5`, not `2`
- `%` is the one you reach for constantly: even/odd, wrapping, leftovers

Remainder Is More Useful Than It Looks

```
n % 2 === 0;           // is n even?  
1000 % 60;             // 40 - leftover seconds  
(i + 1) % items.length; // wrap to the start of an array
```

- Splitting a total into units is `%` for the leftover, `/` for the whole

```
const total = 1000;  
const seconds = total % 60;           // 40  
const minutes = (total - seconds) / 60; // 16
```

Assignment and Increment

```
let n = 10;  
  
n += 5; // 15    same as n = n + 5  
n -= 3; // 12  
n *= 2; // 24  
n /= 4; // 6  
n %= 4; // 2  
  
n++;    // 3    add one  
n--;    // 2    subtract one
```

- `n++` and `n += 1` do the same thing in a statement of their own

Comparison: `===` versus `==`

```
5 === 5;    // true
5 === "5";  // false - different types, no conversion

5 == "5";   // true  - the string was converted first
0 == "";    // true  - both converted to 0
false == 0; // true
```

- `===` compares value *and* type. Use it everywhere.
- `==` converts first, which produces results nobody predicts
- The same applies to `!==` versus `!=`

Converting Types Explicitly

```
Number("42");    // 42
Number("hello");  // NaN (Not-a-Number)
Number("");       // 0    - a classic surprise

String(42);       // "42"
Boolean(0);       // false
Boolean("");      // false
Boolean("hello"); // true
```

- Converting yourself makes code clearer and easier to debug
- Never rely on `==` to do the conversion for you

Building Strings with

```
const first = "Ada";  
const last = "Lovelace";  
  
first + " " + last; // "Ada Lovelace"
```

-  means **add** for numbers and **join** for strings
- If either side is a string, the other is converted to one

```
"Total: " + 30; // "Total: 30"  
"5" + 3;        // "53"    - not 8!  
"5" - 3;        // 2       - only + is overloaded
```

Template Literals

- The modern way to assemble text with embedded values

```
const name = "Alice";
const count = 3;

// Old way:
"Hi " + name + ", you have " + count + " orders.";

// Modern way - backticks, not quotes:
`Hi ${name}, you have ${count} orders.`;
// "Hi Alice, you have 3 orders."
```

- Backticks also span multiple lines without `\n`

`${}` Holds Any Expression

```
const qty = 3;
const price = 4;

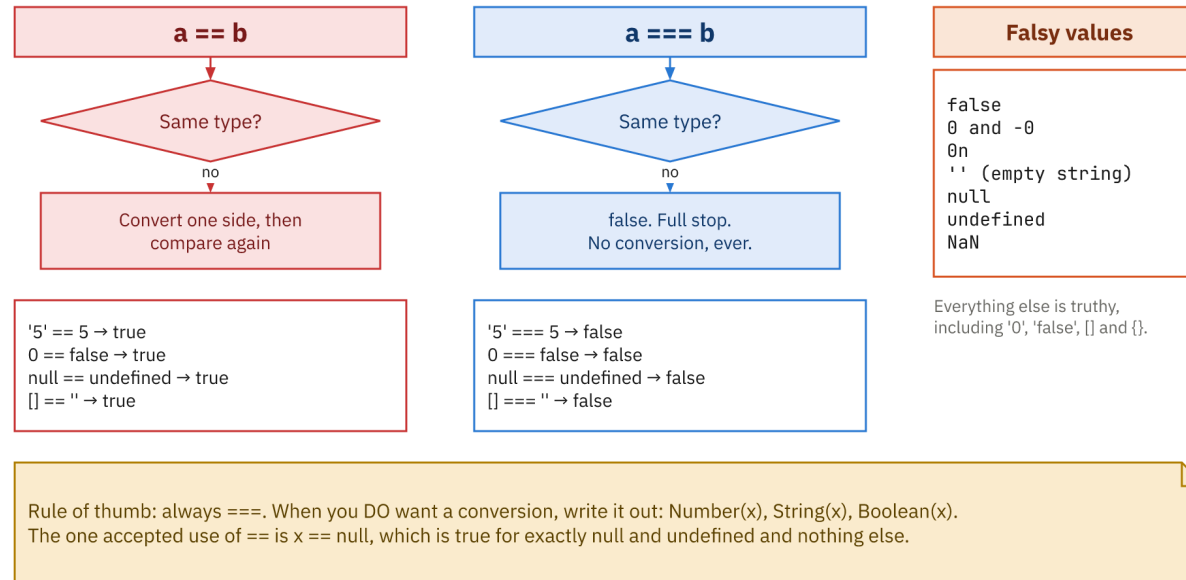
`${qty} bags = $$${qty * price}`; // "3 bags = $12"
`Tax: ${price * 0.07}.toFixed(2)}`; // "Tax: 0.28"
`You are ${age >= 18 ? "an adult" : "a minor"}`;
```

- Not just variables, any expression that produces a value
- Prefer template literals over `+` once there is more than one value

Coercion and Equality

Coercion and Equality

`==` converts before it compares. `===` never converts. Use `===` and convert on purpose.



- `==` converts before it compares; `===` never does
- Always `===`, and convert on purpose with `Number()` or `String()`

Objects

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Object Literals

- A collection of related data and behavior as **key/value pairs**

```
const user = {  
  name: "Alice",  
  age: 30,  
  isAdmin: false,  
};  
  
console.log(user); // { name: 'Alice', age: 30, isAdmin: false }
```

- Values can be any type; pairs separated by commas
- An empty object is just `{}`

Property Access: Dot vs Bracket

```
const user = { name: "Alice", "favorite color": "blue" };

console.log(user.name); // "Alice" (dot notation)

const key = "name";
console.log(user[key]);           // "Alice" (key from a variable)
console.log(user["favorite color"]); // "blue" (space in the key)

console.log(user.email); // undefined (missing - no error)
```


Add, Update, Delete

```
const user = { name: "Alice" };

user.age = 30;           // add
user.name = "Alice S."; // update
delete user.age;         // delete

console.log(user); // { name: 'Alice S.' }
```

- Objects are **mutable**: `const` blocks reassignment, not mutation

```
const u = { name: "Alice" };
u.name = "Bob"; // OK - changing a property
// u = {};      // Error - reassigning the const variable
```

Nested Objects and Optional Chaining

```
const user = {  
  name: "Alice",  
  address: { city: "Seattle", zip: "98101" },  
};  
console.log(user.address.city); // "Seattle"
```

```
const bob = { name: "Bob" }; // no address  
console.log(bob.address.city); // TypeError  
console.log(bob.address?.city); // undefined (no error)
```

- Optional chaining `?.` short-circuits the **whole** rest of the chain

Defaults with ??

- `??` falls back only for `null` / `undefined`
- `||` falls back for **any** falsy value, including valid `0` and `""`

```
const settings = { fontSize: 0, title: "" };

settings.fontSize || 16; // 16  WRONG - 0 was a real choice
settings.fontSize ?? 16; // 0   right

settings.title || "Untitled"; // "Untitled"  WRONG
settings.title ?? "Untitled"; // ""          right

user.address?.city ?? "Unknown"; // the pair, together
```

Shorthand, Computed Keys, Methods

```
const name = "Alice", age = 30;  
const b = { name, age }; // shorthand properties  
  
const field = "email";  
const user = { name, [field]: "alice@example.com" }; // computed key
```

```
const calculator = {  
  value: 0,  
  add(n) { this.value += n; }, // method shorthand  
};  
calculator.add(5);  
console.log(calculator.value); // 5
```

Iterating Over an Object

```
const scores = { math: 90, science: 85, art: 95 };

Object.keys(scores);    // [ 'math', 'science', 'art' ]
Object.values(scores);  // [ 90, 85, 95 ]
Object.entries(scores); // [ ['math', 90], ... ]

for (const [subject, score] of Object.entries(scores)) {
  console.log(`${subject}: ${score}`);
}
```

Checking for a Property

```
const user = { name: "Alice", nickname: undefined };

Object.hasOwn(user, "name");    // true
Object.hasOwn(user, "email");   // false
Object.hasOwn(user, "nickname"); // true - it EXISTS, just undefined

user.email === undefined;      // true, but so is nickname
"name" in user;                 // true (also searches the prototype)
```

- `Object.hasOwn` is the modern replacement for `hasOwnProperty`

Copying, Merging, Freezing

```
const defaults = { theme: "light", fontSize: 14 };
const custom = { fontSize: 18 };

const settings = { ...defaults, ...custom }; // spread
// { theme: 'light', fontSize: 18 } - later sources win
```

- Spread and `Object.assign` make **shallow** copies (nested is shared)
- `structuredClone(obj)` deep-copies plain data
- `Object.freeze(obj)` makes it read-only

Value vs Reference

Value vs Reference

Primitives copy the value. Objects copy the reference: two names, one object.

Primitives copy the VALUE

```
let a = 10;  
let b = a; // copies the number  
b = 20;  
// a is still 10
```

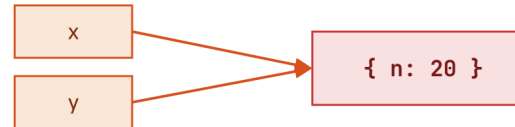
a → 10

b → 20

Two separate boxes. Changing one cannot touch the other. Same for string, boolean, number, null, undefined, symbol, bigint.

Objects copy the REFERENCE

```
const x = { n: 10 };  
const y = x; // copies the arrow, not the box  
y.n = 20;  
// x.n is now 20 too
```



One object, two names. Arrays and functions are objects too, so they behave the same way.

To get a real copy, spread it: `const z = { ...x }` or `const arr2 = [ ...arr ]`
But spread is SHALLOW: a nested object is still shared. Use `structuredClone(x)` for a deep copy.
Also why `const` does not mean immutable: it fixes the arrow, not the box it points at.

- Primitives copy the value; objects copy the reference
- Spread gives a **shallow** copy; a nested object is still shared

Conditionals

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

if, else, else if

```
let score = 75;

if (score >= 90) {
  console.log("Grade: A");
} else if (score >= 80) {
  console.log("Grade: B");
} else if (score >= 70) {
  console.log("Grade: C");
} else {
  console.log("Grade: D or below");
}
```

- The **first** condition that is `true` runs; the rest are skipped
- Order matters: put the narrowest test first

Truthy and Falsy

- `if` checks whether a value is **truthy** or **falsy**
- Falsy values: `false`, `0`, `""`, `null`, `undefined`, `NaN`
- Everything else is **truthy**, including `[]` and `{}`

```
let name = "";  
if (name) {  
  console.log("Name is set.");  
} else {  
  console.log("Name is empty or missing."); // this runs  
}
```

Logical Operators

```
const age = 20, hasTicket = true;

age >= 18 && hasTicket; // true - AND: both must be true
age < 18 || hasTicket;  // true - OR: either will do
!hasTicket;             // false - NOT: flips it
```

- They **short-circuit**: `&&` stops at the first falsy, `||` at the first truthy

```
user && user.name; // safe - stops if user is missing
name || "Anonymous"; // a default (but prefer ?? - see Objects)
```

The `switch` Statement

```
let color = "green";

switch (color) {
  case "red":
    console.log("Stop!");
    break;
  case "green":
    console.log("Go!");
    break;
  default:
    console.log("Unknown color");
}
```

- `case` labels checked top to bottom; `break` stops fall-through
- `default` runs when no case matches

The Ternary Operator

- An **expression** that chooses between two values
- Syntax: `condition ? valueIfTrue : valueIfFalse`

```
let age = 20;  
let message = age >= 18 ? "Adult" : "Minor";  
console.log(message); // "Adult"
```

- Handy for short, simple choices, especially when assigning a value
- Do not nest them; that is what `if / else if` is for

Guard Clauses Beat Nesting

```
// Deeply nested - hard to follow
function pay(user) {
  if (user) {
    if (user.active) {
      if (user.balance > 0) { return "paid"; }
    }
  }
}

// Guard clauses - handle the exits first, then the real work
function pay(user) {
  if (!user) return "no user";
  if (!user.active) return "inactive";
  if (user.balance <= 0) return "no funds";
  return "paid";
}
```

Loops

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

The **for** Loop

- Use when you **know how many times** to repeat, or need the index

```
for (let i = 1; i <= 5; i++) {  
  console.log("Count:", i);  
}  
// Count: 1 ... Count: 5
```

- `let i = 1` → start · `i <= 5` → condition · `i++` → update each pass
- All three parts are optional, but you rarely want to omit them

Counting Backwards

```
const items = ["a", "b", "c"];

for (let i = items.length - 1; i >= 0; i--) {
  console.log(items[i]); // c, b, a
}
```

- Reverse order is where an index-based `for` still beats `for...of`
- So is removing items while looping; go backwards so the indexes behind you do not shift

while and do...while

```
let count = 1;
while (count <= 3) {
  console.log("Loop:", count);
  count++; // change the variable or loop forever!
}
```

```
let number = 0;
do {
  console.log("Number is:", number);
  number++;
} while (number < 3);
```

- `do...while` always runs the body **at least once**

for...of and for...in

```
const fruits = ["apple", "banana", "orange"];
for (const fruit of fruits) {
  console.log(fruit); // each value, no indexes
}
```

```
const user = { name: "Sam", age: 25 };
for (const key in user) {
  console.log(key, "=", user[key]); // keys of an object
}
```

- `for...of` for array **values**; `for...in` for object **keys**
- Do not use `for...in` on an array; you get index *strings*

break and continue

```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) continue; // skip the rest of this pass  
  if (i === 5) break;    // stop the loop entirely  
  console.log(i);  
}  
// 1  
// 2  
// 4
```

- `continue` skips one pass; `break` leaves the loop for good

Choosing a Loop

Need	Reach for
Every value in an array	<code>for...of</code>
The index too	<code>for</code>
Reverse order	<code>for</code> counting down
Unknown number of passes	<code>while</code>
Always run at least once	<code>do...while</code>
Every key of an object	<code>Object.entries</code> + <code>for...of</code>

- Transforming or filtering? Use `map` / `filter` instead; next pair

Arrays

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

What Is an Array?

- A special object that stores an **ordered list** of values
- Common uses: names, scores, todos, cart items
- Values can be mixed types, even functions

```
const empty = [];           // empty
const numbers = [10, 20, 30]; // numbers
const names = ["Sam", "Alex", "Jo"]; // strings
const mixed = [1, "hello", true, { id: 1 }, [2, 3]];
```


Indexes and Length

- Items are numbered from **0** (zero-based)
- `.length` gives the count; last item is `length - 1`

```
const colors = ["red", "green", "blue"];

console.log(colors[0]);    // "red"
colors[1] = "yellow";      // change a value
console.log(colors.length); // 3
console.log(colors[99]);    // undefined - not an error
```

Adding and Removing Items

- End: `push` (add) / `pop` (remove)
- Start: `unshift` (add) / `shift` (remove)

```
const animals = ["cat", "dog"];

animals.push("bird");           // returns the new LENGTH → 3
animals.pop();                  // returns the ITEM removed → "bird"
animals.unshift("mouse");       // ["mouse","cat","dog"]
animals.shift();                 // returns "mouse"
```

- `unshift` / `shift` re-index everything, so they cost more than `push` / `pop`

splice: Insert, Remove, Replace

- `splice(start, deleteCount, ...itemsToInsert)`: all three jobs, in place

```
const items = ["a", "b", "c", "d"];

items.splice(1, 0, "x");    // insert, remove nothing
// ["a", "x", "b", "c", "d"]

items.splice(2, 1);         // remove 1 at index 2 → returns ["b"]
// ["a", "x", "c", "d"]

items.splice(1, 2, "z");    // remove 2, insert 1 → returns ["x", "c"]
// ["a", "z", "d"]
```

- It **returns what it removed** and **mutates** the original

Finding Items

- `includes` → true/false
- `indexOf` → position, or `-1` if not found

```
const letters = ["a", "b", "c", "b"];

letters.includes("b"); // true
letters.includes("z"); // false
letters.indexOf("b");  // 1 (first match)
letters.indexOf("z");  // -1 (not found)
```

- `-1` is truthy, so always compare: `if (arr.indexOf(x) !== -1)`

split and join

- `split` turns a string into an array; `join` turns it back

```
const csv = "red,green,blue";

const colors = csv.split(","); // ["red", "green", "blue"]
colors.join(" | ");           // "red | green | blue"
colors.join("");              // "redgreenblue"

"hello".split("");           // ["h","e","l","l","o"]
```

- They are inverses, and neither changes the original

`slice` Copies, `splice` Mutates

```
const nums = [1, 2, 3, 4, 5];

nums.slice(1, 3); // [2, 3]      - end is EXCLUSIVE
nums.slice(-2);   // [4, 5]      - from the end
nums.slice();     // a full copy
console.log(nums); // [1,2,3,4,5] - untouched

nums.splice(1, 2); // [2, 3] removed
console.log(nums); // [1, 4, 5]  - changed!
```

Looping Over Arrays

- Classic `for` loop uses an index
- `for...of` is simpler when you only need the value

```
const scores = [10, 20, 30];

for (let i = 0; i < scores.length; i++) {
  console.log(scores[i]);
}

for (const score of scores) {
  console.log(score);
}
```

sort: Two Surprises

- It **mutates** the array, and compares as **strings**
- Pass `(a, b) => a - b` for numbers

```
const scores = [10, 9, 100, 1];

scores.sort(); // [1, 10, 100, 9] string order!
scores.sort((a, b) => a - b); // [1, 9, 10, 100]

// Leave the original alone:
[3, 1, 2].toSorted((a, b) => a - b); // [1, 2, 3]
```


Reading From the End: `at()`

- `at(-1)` beats `arr[arr.length - 1]`
- Negative indexes count back from the end

```
const fruits = ["apple", "banana", "orange"];

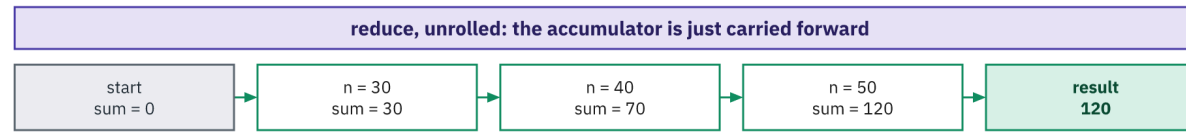
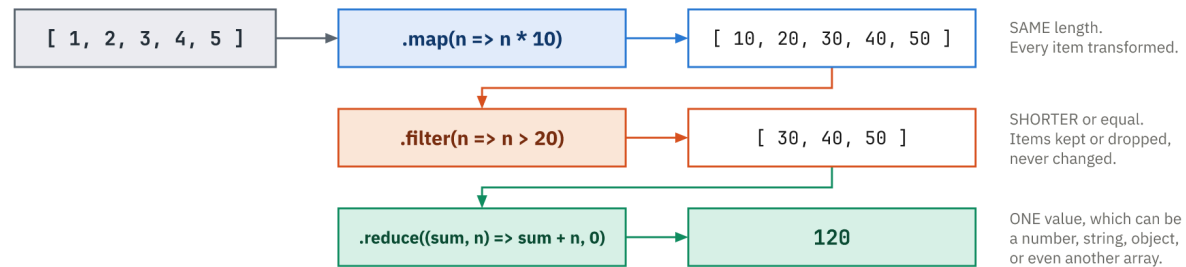
fruits.at(-1); // "orange"
fruits.at(-2); // "banana"
fruits.at(0);  // "apple"

fruits[fruits.length - 1]; // same, but makes you do math
```

map, filter, reduce

map, filter, reduce

Three stages, one data flow. None of them changes the original array; each returns a new value.



Always pass reduce a starting value. Without it, reduce uses the first element as the seed, and throws on an empty array. These return new values, so they chain. `forEach` does not: it returns undefined and exists only for side effects.

- Same length, then shorter, then one value
- None of them changes the original array

Array map, filter, reduce

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

forEach and map

- `forEach` runs a function per item (no new array)
- `map` builds a **new** array of transformed values

```
const nums = [1, 2, 3];

nums.forEach((n, i) => console.log(i, n));

const doubled = nums.map((n) => n * 2);
console.log(doubled); // [2, 4, 6]
console.log(nums);    // [1, 2, 3] - unchanged
```

- `map` always returns an array the **same length** as the original

filter

- Keeps only the items for which your function returns `true`
- Returns a **new**, usually shorter, array

```
const ages = [15, 21, 18, 30];
const adults = ages.filter((age) => age >= 18);
console.log(adults); // [21, 18, 30]

const users = [
  { name: "Sam", age: 25 },
  { name: "Alex", age: 30 },
];
users.filter((u) => u.age > 27); // [{ name: "Alex", age: 30 }]
```

reduce: Many Values to One

- `map` / `filter` return arrays; `reduce` returns **one value**
- Takes a function **and a starting value**; always pass the seed

```
const prices = [10, 20, 30];

const total = prices.reduce((sum, price) => sum + price, 0);
//                        ^running ^current    ^start

console.log(total); // 60
// 0 + 10 = 10, 10 + 20 = 30, 30 + 30 = 60
```

Always Pass the Seed

```
[].reduce((a, b) => a + b, 0); // 0 - fine  
  
// [].reduce((a, b) => a + b);  
// TypeError: Reduce of empty array with no initial value
```

- Without a seed, `reduce` uses the **first item** as the starting value
- That breaks on an empty array, and changes the type of the result
- The seed also declares what you are building: `0`, `" "`, `[]`, `{}`

reduce: Building an Object

- The accumulator can be any type, not just a number
- Here it tallies how often each word appears

```
const words = ["js", "css", "js"];

const counts = words.reduce((tally, word) => {
  tally[word] = (tally[word] || 0) + 1;
  return tally;
}, {});

console.log(counts); // { js: 2, css: 1 }
```


find, some, and every

- `find` → the **first** match, or `undefined`
- `some` / `every` → a plain boolean

```
const nums = [4, 9, 16, 25];

nums.find((n) => n > 10); // 16
nums.find((n) => n > 100); // undefined

nums.some((n) => n > 20); // true - is ANY over 20?
nums.every((n) => n > 20); // false - are ALL over 20?
```

- Use `find` for one item, `filter` for a list

Chaining Them Together

- Each returns an array, so they compose left to right

```
const orders = [
  { item: "mug",    qty: 2, price: 12 },
  { item: "kettle", qty: 1, price: 40 },
  { item: "spoon",  qty: 8, price: 3  },
];

const bigTotal = orders
  .filter((o) => o.qty > 1)           // mug, spoon
  .map((o) => o.qty * o.price)       // [24, 24]
  .reduce((sum, n) => sum + n, 0);   // 48
```

Choosing the Right One

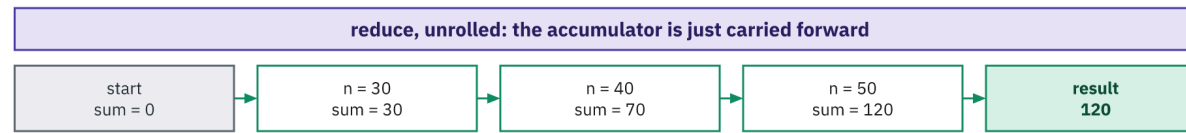
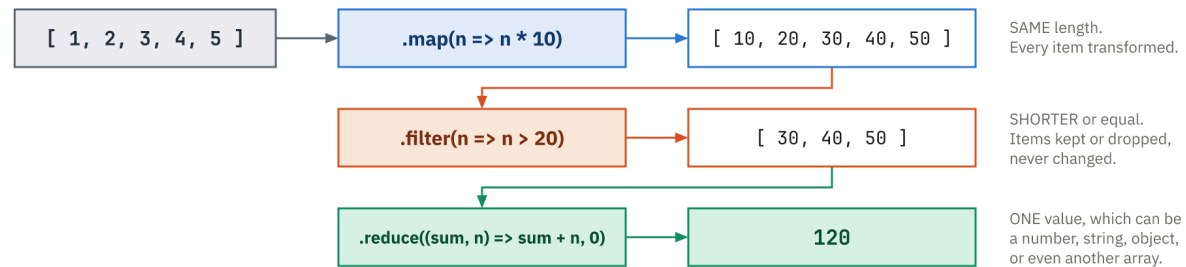
You want	Use
A new array, same length	<code>map</code>
A shorter array	<code>filter</code>
One value out of many	<code>reduce</code>
The first matching item	<code>find</code>
A yes/no answer	<code>some</code> / <code>every</code>
Just side effects	<code>forEach</code>

- If you are `push`ing inside a `forEach`, you probably wanted `map` or `filter`

The Pipeline, End to End

map, filter, reduce

Three stages, one data flow. None of them changes the original array; each returns a new value.



Always pass reduce a starting value. Without it, reduce uses the first element as the seed, and throws on an empty array. These return new values, so they chain. `forEach` does not: it returns undefined and exists only for side effects.

- Each stage returns a new value, which is why they chain
- `forEach` returns `undefined`, so it cannot

Functions

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

What Is a Function?

- A reusable, named block of code
- Optional inputs, optional returned result
- Avoids repetition; organizes logic

```
function greet() {  
  console.log("Hello!");  
}  
  
greet(); // call (invoke) the function
```

Parameters and Arguments

- **Parameters** are names in the definition
- **Arguments** are the values you pass
- Give parameters **default values** for missing args

```
function greetUser(name = "guest") {  
  console.log("Welcome, " + name + "!");  
}  
  
greetUser("Sam"); // "Welcome, Sam!"  
greetUser();      // "Welcome, guest!"
```

- A missing argument is `undefined`; the default fills in for exactly that

Return Values

- `return` sends a value back and **stops** the function
- No `return` means the result is `undefined`

```
function add(a, b) {  
  return a + b;  
}  
  
const result = add(3, 4);  
console.log(result); // 7
```

- Everything after a `return` in that branch is dead code

Function Expressions

- A function can be stored in a variable
- Useful for passing functions around

```
const multiply = function (a, b) {  
  return a * b;  
};  
  
console.log(multiply(2, 5)); // 10
```

Arrow Functions

- Shorter syntax for function expressions
- A single expression can drop `{}` and `return`

```
const divide = (a, b) => {  
  return a / b;  
};  
  
const square = (n) => n * n;    // implicit return  
  
console.log(divide(10, 2)); // 5  
console.log(square(4));     // 16
```

Declarations Hoist, Expressions Do Not

```
sayHi();           // works - the whole function is hoisted
function sayHi() { console.log("hi"); }
```



```
sayBye();          // ReferenceError: Cannot access before initialization
const sayBye = () => console.log("bye");
```

- A **declaration** is available everywhere in its scope
- An **expression** only exists after the assignment line runs
- This is why "define before use" is a safe habit either way

Rest Parameters

- `...` collects every remaining argument into a real array

```
function sumAll(...numbers) {  
  return numbers.reduce((total, n) => total + n, 0);  
}
```

```
sumAll(1, 2, 3);           // 6  
sumAll(10, 20, 30, 40);   // 100  
sumAll();                  // 0
```

- Must be the **last** parameter
- Unlike the old `arguments` object, it is a genuine array

Spreading Arguments

- The same `...` at a **call site** unpacks an array into arguments

```
const prices = [5, 10, 15, 20];

sumAll(prices);           // "05,10,15,20" - ONE argument, the array itself,
                          //   which `0 + [...]" quietly turns into a string
sumAll(...prices);        // 50 - four separate arguments

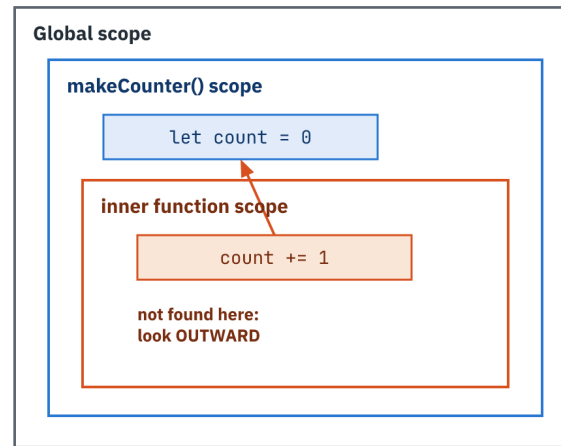
Math.max(...prices);      // 20   (Math.max(prices) would be NaN)
```

- Rest **collects** in a definition; spread **expands** in a call
- Forgetting the `...` rarely errors; it just gives you nonsense

The Scope Chain

Scope Chain and Closures

Lookup goes OUTWARD only. A closure is the link that survives after the outer call returns.



Lookup never goes inward: the outer scope cannot see anything declared inside, and two sibling scopes cannot see each other.

After makeCounter() returns

The makeCounter() frame is gone from the stack, but the returned function still holds its scope link, so count is kept alive and stays private.

```
const a = makeCounter();  
const b = makeCounter();  
a(); a(); // 1, 2  
b(); // 1 - its own count
```

Every call to the outer function makes a NEW scope, so a and b capture two independent counts. That is how closures give you private state without a class.

- Lookup goes **outward only**, never inward
- A closure is the link that survives the outer call returning

Destructuring, Rest and Spread

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Object Destructuring

- Extract properties into variables
- Rename with `:`, supply defaults with `=`

```
const user = { name: "Sam", age: 25, isAdmin: false };

const { name, age } = user;
const { name: userName } = user;    // rename
const { role = "member" } = user;   // default

console.log(name, age, userName, role);
// "Sam" 25 "Sam" "member"
```

- Matches by **name**, so order does not matter

Destructuring in Parameters

- Very common: pull fields straight from an argument object

```
function greet({ name, age }) {  
  console.log("Hello " + name + ", age " + age);  
}  
  
greet({ name: "Sam", age: 25 });  
// "Hello Sam, age 25"
```

- The call site reads like named arguments, in any order
- Add defaults right there: `function greet({ name, age = 0 })`

Array Destructuring

- Uses **position**, not property names
- Skip items with commas; defaults with `=`

```
const colors = ["red", "green", "blue"];

const [first, second] = colors; // "red", "green"
const [primary, , tertiary] = colors; // skip green

let x = 1, y = 2;
[x, y] = [y, x]; // swap → 2 1
```

Rest in Destructuring

- `...` **collects** the leftover items
- Works for arrays and objects

```
const scores = [10, 20, 30, 40];  
const [top, ...others] = scores;  
console.log(others); // [20, 30, 40]  
  
const user = { id: 1, name: "Sam", age: 25 };  
const { id, ...rest } = user;  
console.log(rest); // { name: "Sam", age: 25 }
```

- Must come last; `[...rest, last]` is a syntax error

Spread for Arrays

- `...` **expands** an array's items
- Copy, merge, insert, or pass as arguments

```
const a = [1, 2];  
const b = [3, 4];  
  
const copy = [...a];           // [1, 2]  
const merged = [...a, ...b];  // [1, 2, 3, 4]  
const inserted = [0, ...a, 9]; // [0, 1, 2, 9]  
  
function sum(x, y, z) { return x + y + z; }  
console.log(sum(...[5, 10, 15])); // 30
```

Spread for Objects

- Copy and merge object properties
- Later spreads **override** earlier ones

```
const baseUser = { name: "Sam", age: 25 };
const extra = { age: 26, isAdmin: true };

const merged = { ...baseUser, ...extra };
console.log(merged);
// { name: "Sam", age: 26, isAdmin: true }
```

- This is the idiomatic "copy and change one field"

Spread Copies Are Shallow

```
const original = { name: "Sam", address: { city: "Seattle" } };
const copy = { ...original };

copy.name = "Alex";           // safe - top level is a real copy
copy.address.city = "Boston"; // NOT safe - same nested object!

console.log(original.address.city); // "Boston"
```

- Nested objects are **shared**, not duplicated
- Use `structuredClone(original)` for a genuine deep copy

Rest vs Spread

- Same `...`, meaning depends on **context**
- **Rest**: collects, in destructuring / parameters
- **Spread**: expands, in literals / function calls

```
const [head, ...tail] = [1, 2, 3]; // rest → collect  
const clone = [...tail];           // spread → expand
```

- Left of the `=`, or in a parameter list → rest
- Inside `[]`, `{}`, or a call → spread

The Shapes, Side by Side

Destructuring, Rest and Spread

The pattern on the left mirrors the shape on the right. Three dots collect on the left, and expand on the right.

Object destructuring: matched by NAME	Array destructuring: matched by POSITION
<pre>const user = { id: 7, name: 'Ada', city: 'Bath', tz: 'GMT' };</pre>	<pre>const scores = [90, 80, 70, 60];</pre>
<pre>const { name, city: town, role = 'guest', ...rest } = user;</pre>	<pre>const [first, , third, ...others] = scores;</pre>
name = 'Ada' → same key, plucked out town = 'Bath' → renamed with a colon role = 'guest' → default, the key was missing rest = { id: 7, tz: 'GMT' } → everything left over	first = 90 → position 0 (the empty slot skips position 1) third = 70 → position 2 others = [60] → rest must come LAST
Same three dots, opposite jobs	
REST: on the left of the equals Gathers what is left into one new value. <pre>const [head, ...tail] = list;</pre><pre>function sum(...numbers) { }</pre>	SPREAD: on the right, or in a call Unpacks one value into many. <pre>const copy = { ...user, city: 'Leeds' };</pre><pre>const merged = [...a, ...b];</pre>
Spreading copies only ONE level deep: a nested object is still shared with the original. Later keys win, which is how { ...defaults, ...options } works. And destructuring a function parameter, function f({ id, name }), gives named arguments.	

- Objects match by **name**, arrays match by **position**
- Three dots collect on the left, and expand on the right

Closures

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Lexical Scope Recap

- Scope = the variables a piece of code can "see"
- JS uses **lexical scope**: based on where code is **written**
- Inner functions can read outer variables, never the reverse

```
function outer() {  
  const message = "hello";  
  function inner() {  
    console.log(message); // sees outer's variable  
  }  
  inner();  
}  
outer(); // "hello"
```

What Is a Closure?

- A function **remembers** variables from where it was created
- Survives after the outer function has returned
- Each call captures a **separate** value

```
function makeGreeter(name) {  
  return function () {  
    console.log("Hello, " + name + "!");  
  };  
}  
  
const greetAlice = makeGreeter("Alice");  
greetAlice(); // "Hello, Alice!"
```

The Classic Counter

- Inner function keeps a **private, persistent** variable
- `count` is not global and not on any object

```
function makeCounter() {  
  let count = 0;  
  return function () {  
    count++;  
    return count;  
  };  
}  
  
const counter = makeCounter();  
console.log(counter(), counter(), counter()); // 1 2 3
```

Each Closure Gets Its Own Scope

```
const a = makeCounter();  
const b = makeCounter();  
  
a(); a(); // 1, 2  
b();      // 1 - completely independent of a
```

- Every call to `makeCounter` creates a **new** `count`
- This is why closures work as a factory for isolated state

Private State

- Closures give **true private state**
- Only the exposed functions can change the data

```
function createAccount(start) {  
  let balance = start;  
  return {  
    deposit(n) { balance += n; return balance; },  
    getBalance() { return balance; },  
  };  
}  
  
const acct = createAccount(100);  
console.log(acct.deposit(50)); // 150  
console.log(acct.balance);     // undefined - unreachable
```

The `var`-in-a-loop Pitfall

- `var` has **one shared** binding for the whole loop
- Callbacks all see the final value

```
const funcs = [];  
for (var i = 0; i < 3; i++) {  
  funcs.push(() => i);  
}  
console.log(funcs[0]()); // 3 (not 0!)
```

- The loop finished before any callback ran, and `i` is 3 by then

Fix With `let`

- `let` creates a **new binding per iteration**
- Each closure captures its own value

```
const funcs = [];  
for (let i = 0; i < 3; i++) {  
  funcs.push(() => i);  
}  
console.log(funcs[0](), funcs[1](), funcs[2]()); // 0 1 2
```

- One more reason `let` replaced `var`

Practical Pattern: memoize

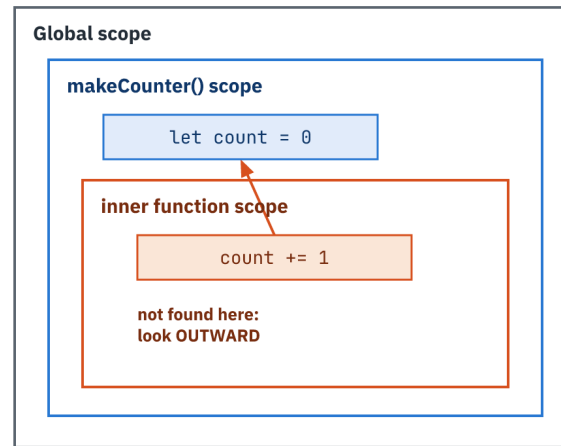
- Private `cache` persists across calls
- Reuses stored answers instead of recomputing

```
function memoize(fn) {  
  const cache = new Map();  
  return function (n) {  
    if (cache.has(n)) return cache.get(n);  
    const result = fn(n);  
    cache.set(n, result);  
    return result;  
  };  
}  
  
const fast = memoize((n) => n * n);  
console.log(fast(4)); // computes → 16  
console.log(fast(4)); // cached   → 16
```

What a Closure Actually Is

Scope Chain and Closures

Lookup goes OUTWARD only. A closure is the link that survives after the outer call returns.



Lookup never goes inward: the outer scope cannot see anything declared inside, and two sibling scopes cannot see each other.

After makeCounter() returns

The makeCounter() frame is gone from the stack, but the returned function still holds its scope link, so count is kept alive and stays private.

```
const a = makeCounter();
const b = makeCounter();
a(); a(); // 1, 2
b(); // 1 - its own count
```

Every call to the outer function makes a NEW scope, so a and b capture two independent counts. That is how closures give you private state without a class.

- The frame is gone from the stack, but the scope link survives
- Every call makes a new scope, so each closure gets its own copy

Classes and Encapsulation

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

What Is a Class?

- A **blueprint** for creating objects
- Describes an object's **data** (properties) and **behavior** (methods)
- Great for many similar objects: users, products, characters

```
class Person {  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  sayHello() {  
    console.log("Hi, my name is " + this.name);  
  }  
}
```

Constructor, `this`, and `new`

- `new` creates an **instance** and runs the constructor
- `this` refers to the specific object being built
- Each instance gets its own properties

```
const alice = new Person("Alice", 25);  
const bob = new Person("Bob", 30);  
  
alice.sayHello(); // "Hi, my name is Alice"  
bob.sayHello();   // "Hi, my name is Bob"
```

Methods Are Shared

```
const alice = new Person("Alice", 25);  
const bob = new Person("Bob", 30);  
  
console.log(alice.sayHello === bob.sayHello); // true
```

- Properties are **per instance**; methods live on the prototype
- One function object, however many instances you create
- That is why methods are cheap and fields are not

Class Fields

- Declare properties with default values, outside the constructor

```
class Player {  
  score = 0;    // field with default  
  lives = 3;  
  
  constructor(name) { this.name = name; }  
  addScore(points) { this.score += points; }  
}  
  
const p = new Player("Sam");  
p.addScore(10);  
console.log(p.score); // 10
```

#private Fields

- A `#` prefix makes a field genuinely private
- Unreachable from outside the class body; not a convention, a rule

```
class BankAccount {  
  #balance;  
  
  constructor(start) { this.#balance = start; }  
  get balance() { return this.#balance; }  
}  
  
const acct = new BankAccount(100);  
console.log(acct.balance); // 100  
// console.log(acct.#balance); // SyntaxError  
console.log(Object.keys(acct)); // [] - invisible
```


Getters and Setters

- Look like properties from outside, but run code

```
class Person {  
  #first;  
  
  constructor(first) { this.#first = first; }  
  
  get first() { return this.#first; }  
  
  set first(value) {  
    if (typeof value !== "string" || value.trim() === "") {  
      throw new Error("Name must be a non-empty string");  
    }  
    this.#first = value;  
  }  
}
```

- No parentheses at the call site: `p.first = "Ada"`

Validation Belongs in the Setter

```
const p = new Person("John");

p.first = "Jane"; // fine
p.first = "";     // Error: Name must be a non-empty string
```

- Because `#first` is private, **nothing can bypass the setter**
- A plain public property has no such guarantee
- This is the whole point of encapsulation: one place to enforce a rule

Computed Getters

- A getter can derive a value instead of storing one

```
class Employee {  
  #first; #last; #salary;  
  
  constructor(f, l, s) { this.#first = f; this.#last = l; this.#salary = s; }  
  
  get fullName() { return `${this.#first} ${this.#last}`; }  
  get salaryFormatted() { return "$" + this.#salary.toFixed(2); }  
}  
  
const e = new Employee("Jane", "Doe", 82500);  
console.log(e.fullName);           // "Jane Doe"  
console.log(e.salaryFormatted);    // "$82500.00"
```

Where Each Part Lives

Class Anatomy

Where each part actually lives: fields on the instance, methods on the prototype, statics on the class itself.

```
class Animal {
  static kingdom = 'Animalia';
  #secret = 'hidden';
  constructor(name) { this.name = name; }
  speak() { return this.name + ' makes a sound'; }
  get label() { return this.name.toUpperCase(); }
}
class Dog extends Animal {
  constructor(name, breed) {
    super(name);
    this.breed = breed;
  }
  speak() { return super.speak() + ' (a bark)'; }
}
```

class is syntax over the prototype chain, not a new object model: the same lookup rules apply. Two things it does add: #private fields are enforced by the language (unlike a leading underscore), and the whole class body runs in strict mode.

On the INSTANCE

name, breed
#secret
One copy per object. Anything assigned to this in the constructor.

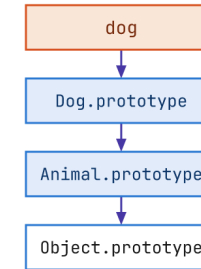
On the PROTOTYPE

speak(), get label
ONE copy, shared by every instance. This is why methods cost nothing per object.

On the CLASS

Animal.kingdom
Statics belong to the class, not to any instance. dog.kingdom is undefined.

What extends actually does



extends just links these. super(name) runs the parent constructor, and you must call it before touching this.

- Fields on the instance, methods on the prototype, statics on the class
- `class` is syntax over the prototype chain, not a new object model

Class Inheritance, Overriding and Statics

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Inheritance with `extends`

- `extends` reuses another class's behavior
- `super(...)` calls the parent constructor

```
class Animal {  
  constructor(name) { this.name = name; }  
  speak() { console.log(this.name + " makes a sound."); }  
}  
  
class Dog extends Animal {  
  constructor(name, breed) {  
    super(name);          // must come FIRST  
    this.breed = breed;  
  }  
}  
  
new Dog("Rex", "Lab").speak(); // "Rex makes a sound."
```

`super()` Must Come First

```
class Dog extends Animal {  
  constructor(name, breed) {  
    // this.breed = breed;    // ReferenceError!  
    super(name);  
    this.breed = breed;      // fine, after super  
  }  
}
```

- Until `super()` runs, `this` does not exist yet
- If a subclass has no constructor at all, JS supplies one that just forwards every argument to `super`

Overriding a Method

- Define a method with the same name to replace the parent's

```
class Dog extends Animal {  
  speak() { console.log(this.name + " barks!"); }  
}  
  
new Dog("Rex").speak(); // "Rex barks!"
```

- Lookup walks **up** the chain and stops at the first match
- The parent's version is not deleted, just shadowed

Calling Back Up With `super.method()`

- Extend the parent's behavior instead of replacing it

```
class Dog extends Animal {  
  speak() {  
    console.log("Clearing throat...");  
    super.speak();          // run the parent's version too  
  }  
}  
  
new Dog("Rex").speak();  
// "Clearing throat..."  
// "Rex makes a sound."
```

Static Members

- `static` belongs to the **class**, not to any instance

```
class Vehicle {  
  static registry = "State DMV";  
  static count = 0;  
  
  constructor(make) {  
    this.make = make;  
    Vehicle.count++;  
  }  
}  
  
new Vehicle("Toyota");  
console.log(Vehicle.registry); // "State DMV"  
console.log(Vehicle.count);    // 1
```

Statics Are Not on Instances

```
const v = new Vehicle("Mazda");

console.log(Vehicle.registry); // "State DMV"
console.log(v.registry);       // undefined
```

- Read them off the **class name**, always
- A subclass's `super()` still runs the parent constructor, so a shared counter counts subclass instances too

Static Factory Methods

- A named alternative to `new` with magic arguments

```
class Car extends Vehicle {  
  constructor(make, doors) { super(make); this.doors = doors; }  
  
  static coupe(make) { return new Car(make, 2); }  
}  
  
const c = Car.coupe("Mazda");  
console.log(c.doors); // 2
```

- `Car.coupe("Mazda")` reads better than `new Car("Mazda", 2)`

instanceof Walks the Chain

```
class Bicycle {}

const car = new Car("Toyota", 4);

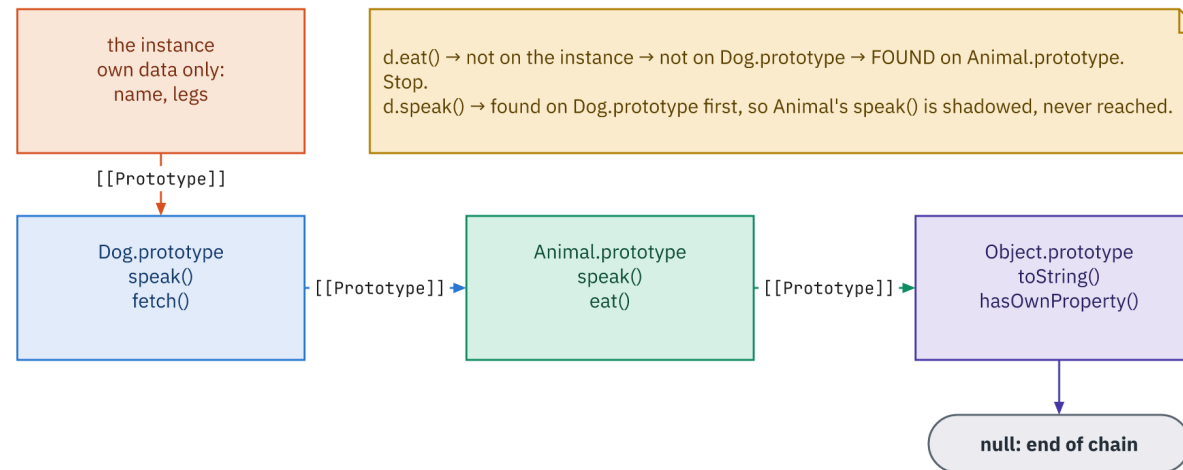
car instanceof Car;    // true
car instanceof Vehicle; // true - up the chain
car instanceof Object; // true - everything ends here
car instanceof Bicycle; // false
```

- `instanceof` asks "is this constructor's prototype anywhere in my chain?"

What **extends** Builds

The Prototype Chain

Property lookup walks UP the chain and stops at the first match. class builds exactly this.



class Dog extends Animal builds this chain for you: extends sets the link, super() runs the parent constructor, and instanceof asks whether a given prototype appears anywhere along the chain.

- Lookup walks **up** until it finds a match, then stops
- **super()** runs the parent constructor before you touch **this**

Call-Site **this** versus Lexical **this**

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Two Big Ideas

- Regular `function` → **call-site** `this` (how it's called)
- Arrow `=>` → **lexical** `this` (where it's defined)
- Choosing correctly avoids most `this` confusion

```
function showThis() { console.log(this); }

const obj = { showThis };
obj.showThis(); // this = obj

const ref = obj.showThis;
ref();          // this = global / undefined
```


Call-Site **this** Patterns

- `obj.method()` → `this` is `obj`
- Detached call → `this` is global / undefined

```
const user = {  
  name: "Alice",  
  sayHi: function () {  
    console.log("Hi, I am", this.name);  
  }  
};  
  
user.sayHi();           // "Hi, I am Alice"  
const greet = user.sayHi;  
greet();                // this.name is undefined
```

Ask the Call Site, Not the Definition

How it is called	<code>this</code> is
<code>new Fn()</code>	the brand-new object
<code>fn.call(x)</code> / <code>.apply</code> / <code>.bind(x)</code>	whatever you passed
<code>obj.method()</code>	<code>obj</code> , left of the dot
<code>fn()</code> bare	<code>undefined</code> (strict) / <code>globalThis</code>
any arrow function	the enclosing scope's <code>this</code>

- Work down the list; the first match wins

The Nested Function Problem

- An inner `function` has its **own** call-site `this`
- Inside `setTimeout` it is **not** the outer object

```
const counter = {  
  value: 0,  
  start: function () {  
    setTimeout(function () {  
      this.value++;           // this is NOT counter  
      console.log(this.value); // wrong  
    }, 1000);  
  }  
};  
  
counter.start();
```

Old Workarounds

- Save `this` in a variable, or `.bind(this)`
- Both work but are wordy and easy to forget

```
start: function () {  
  const self = this;           // capture  
  setTimeout(function () {  
    self.value++;  
  }, 1000);  
  
  setTimeout(function () {  
    this.value++;  
  }.bind(this), 1000);        // or bind  
}
```

Arrow Functions to the Rescue

- Arrows have **no own `this`**; they use the surrounding one
- No `self = this`, no `.bind` needed

```
const counter = {
  value: 0,
  start: function () {
    setTimeout(() => {
      this.value++;           // this is counter
      console.log(this.value);
    }, 1000);
  }
};

counter.start();
```

Arrows in Array Callbacks

- Same fix inside `forEach`, `map`, event listeners
- Arrow reuses the method's `this`

```
const list = {  
  items: ["a", "b", "c"],  
  logAll: function () {  
    this.items.forEach((item) => {  
      console.log(this.items.length, item); // this is list  
    });  
  }  
};  
  
list.logAll();
```

Pitfall: Arrow as a Method

- An arrow **method** captures the outer (often global) `this`
- Use `function` to define methods; arrows for nested callbacks

```
const user = {  
  name: "Alice",  
  sayHi: () => {  
    console.log("Hi", this.name); // NOT user  
  }  
};  
  
user.sayHi(); // this.name is undefined
```

Class Arrow Fields Survive Detaching

- A class field holding an arrow captures `this` at construction

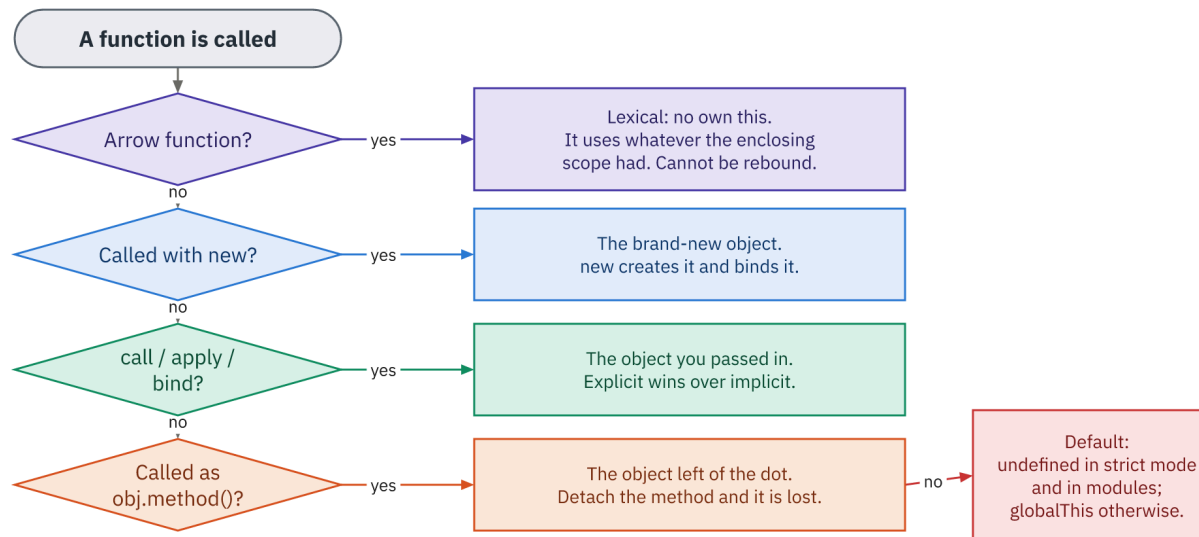
```
class Watch {  
  count = 0;  
  ring = () => {           // arrow FIELD, not a method  
    this.count++;  
    console.log(this.count);  
  };  
}  
  
const w = new Watch();  
const detached = w.ring;  
detached(); // 1 - still bound
```

- Use it for handlers you pass elsewhere, not for every method

The Whole Decision, on One Page

How this Is Decided

Ask the call site, not where the function was written. Work down: the first match wins.



- Work down the list: the first rule that matches wins
- Ask the **call site**, not where the function was written

Constructors and Prototypes

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Function Constructors + `new`

- Pre-`class` blueprint: a capitalized function
- `new` creates an object, links its prototype, runs with `this`, returns it

```
function Person(name, age) {  
  this.name = name;  
  this.age = age;  
}  
  
const alice = new Person("Alice", 30);  
console.log(alice.name); // "Alice"
```

- Forget the `new` and `this` is not the new object, a classic bug

The `.prototype` Property

- Every function has a `prototype` object
- Methods put there are **shared** by all instances (one copy)

```
function Person(name) { this.name = name; }

Person.prototype.greet = function () {
  console.log("Hi, I'm " + this.name);
};

const alice = new Person("Alice");
const bob = new Person("Bob");
console.log(alice.greet === bob.greet); // true - same function
```

The Prototype Chain

- Lookup checks the object, then its prototype, then up to `null`
- Data lives on the instance; behavior on the prototype

```
function Person(name) { this.name = name; }
Person.prototype.greet = function () { return "Hi, " + this.name; };

const alice = new Person("Alice");
console.log(Object.hasOwn(alice, "name")); // true
console.log(Object.hasOwn(alice, "greet")); // false - on prototype
```

Walking the Chain Yourself

```
const alice = new Person("Alice");

Object.getPrototypeOf(alice) === Person.prototype;    // true
Object.getPrototypeOf(Person.prototype) === Object.prototype; // true
Object.getPrototypeOf(Object.prototype);              // null
```

- Three hops and you are at the top
- A property missing from the whole chain is `undefined`, **not** an error

Object.create and instanceof

- `Object.create(proto)` picks a prototype directly, no constructor
- `instanceof` checks if a constructor's prototype is in the chain

```
const animal = { describe() { return this.type + " " + this.name; } };
const dog = Object.create(animal);
dog.type = "dog"; dog.name = "Rex";
console.log(dog.describe()); // "dog Rex"

console.log(dog instanceof Object); // true
```

Own Properties Shadow Inherited Ones

```
const animal = { speak() { return "generic sound"; } };

const cat = Object.create(animal);
cat.speak = function () { return "meow"; }; // own

const dog = Object.create(animal);          // no own speak

console.log(cat.speak()); // "meow"          - own wins
console.log(dog.speak()); // "generic sound" - inherited
console.log(Object.hasOwn(cat, "speak")); // true
console.log(Object.hasOwn(dog, "speak")); // false
```


class Is Syntax Sugar

- `class` uses the **same** constructors and prototypes underneath
- Methods still land on `ClassName.prototype`

```
class Animal2 {  
  constructor(name) { this.name = name; }  
  speak() { return this.name + " makes a sound"; }  
}  
  
const a = new Animal2("Rex");  
console.log(Object.hasOwn(a, "speak")); // false - on prototype  
console.log(a instanceof Animal2);    // true
```

The Mapping, Piece by Piece

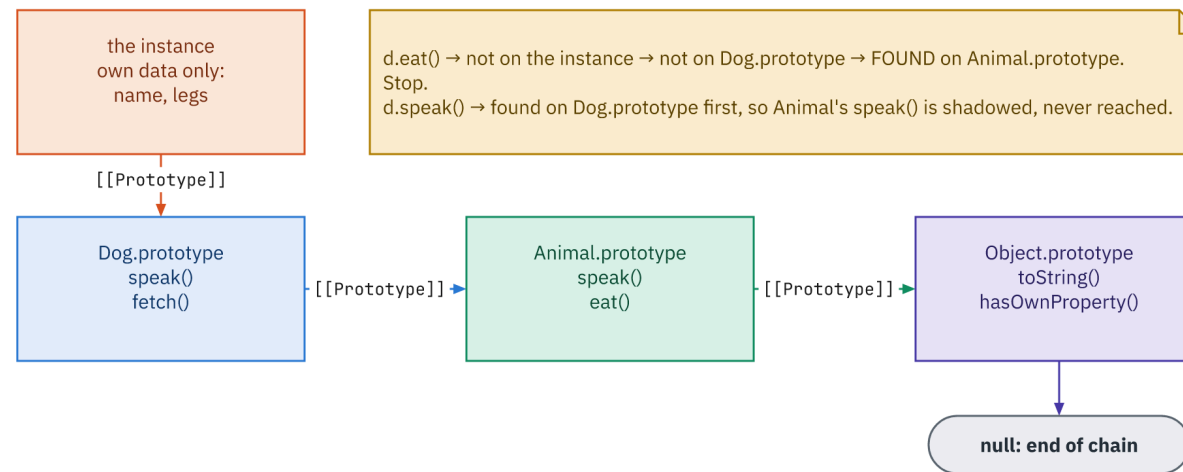
Constructor function	class
<code>function Person(n) { ... }</code>	<code>constructor(n) { ... }</code>
<code>Person.prototype.greet = fn</code>	<code>greet() { ... }</code>
<code>Child.prototype = Object.create(Parent.prototype)</code>	<code>class Child extends Parent</code>
<code>Parent.call(this, args)</code>	<code>super(args)</code>
<code>Person.helper = fn</code>	<code>static helper() { ... }</code>

- Same machinery, far less ceremony, and no chance to forget a step

The Chain a Constructor Builds

The Prototype Chain

Property lookup walks UP the chain and stops at the first match. class builds exactly this.



class `Dog` extends `Animal` builds this chain for you: `extends` sets the link, `super()` runs the parent constructor, and `instanceof` asks whether a given prototype appears anywhere along the chain.

- `[[Prototype]]` links every object up to `Object.prototype`, then `null`
- `instanceof` just asks whether a prototype appears along the chain

Error Handling

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Defensive Checks

- Don't assume DOM elements or data exist
- Missing element returns `null` → guard before use
- Early `return` keeps code flat and clear

```
function setupButton() {  
  const button = document.getElementById("submit");  
  if (!button) {  
    console.warn("Submit button missing; skipping.");  
    return;  
  }  
  button.addEventListener("click", () => console.log("Clicked!"));  
}
```

try / catch / finally

- Wrap risky code so a throw doesn't crash the script
- `catch` handles the error; `finally` always runs

```
try {  
  const data = JSON.parse('{ "name": "Alice", }'); // invalid  
  console.log(data);  
} catch (error) {  
  console.error("Failed to parse:", error.message);  
} finally {  
  console.log("Done (success or fail)");  
}
```

The Error Object

- A thrown error is usually an `Error` object
- Useful fields: `name`, `message`, `stack`
- `TypeError`, `ReferenceError`, etc. handled the same way

```
try {  
  throw new Error("Something went wrong");  
} catch (error) {  
  console.log(error.name);    // "Error"  
  console.log(error.message); // "Something went wrong"  
}
```

Throwing Your Own Errors

- Signal problems with clear, specific messages
- Lets calling code decide what to do

```
function getUsername(user) {  
  if (!user || !user.name) {  
    throw new Error("User object must have a name");  
  }  
  return user.name;  
}  
  
try {  
  getUsername({});  
} catch (error) {  
  console.error(error.message);  
}
```


Always Throw an **Error**

```
throw "Something broke";           // legal, but avoid  
throw new Error("Something broke"); // do this
```

- A string has no `.message`, no `.stack`, and fails `instanceof Error`
- Anything that catches it has to guess what it received
- The same rule applies to `reject(...)` in a promise

Custom Error Types

- Extend `Error` when callers need to tell errors apart
- Carry extra context as properties

```
class ValidationError extends Error {  
  constructor(field, message) {  
    super(message);  
    this.name = "ValidationError";  
    this.field = field;  
  }  
}
```

Catching One Kind of Error

```
try {  
  throw new ValidationError("email", "Email is required");  
} catch (error) {  
  if (error instanceof ValidationError) {  
    console.log(error.field, error.message);  
  } else {  
    throw error;    // not ours - pass it on  
  }  
}
```

- **Re-throw what you did not expect.** A bare `catch` that swallows everything turns a real bug into silence

finally Runs No Matter What

```
function read() {  
  try {  
    return "value";    // returns...  
  } finally {  
    console.log("cleanup"); // ...but this runs FIRST  
  }  
}  
  
console.log(read());  
// "cleanup"  
// "value"
```

- Runs on success, on throw, and even on `return`
- The right home for closing, unlocking, or re-enabling a button

Handling Async Errors

- Promises: attach `.catch` to the chain
- `async` / `await`: use ordinary `try` / `catch`

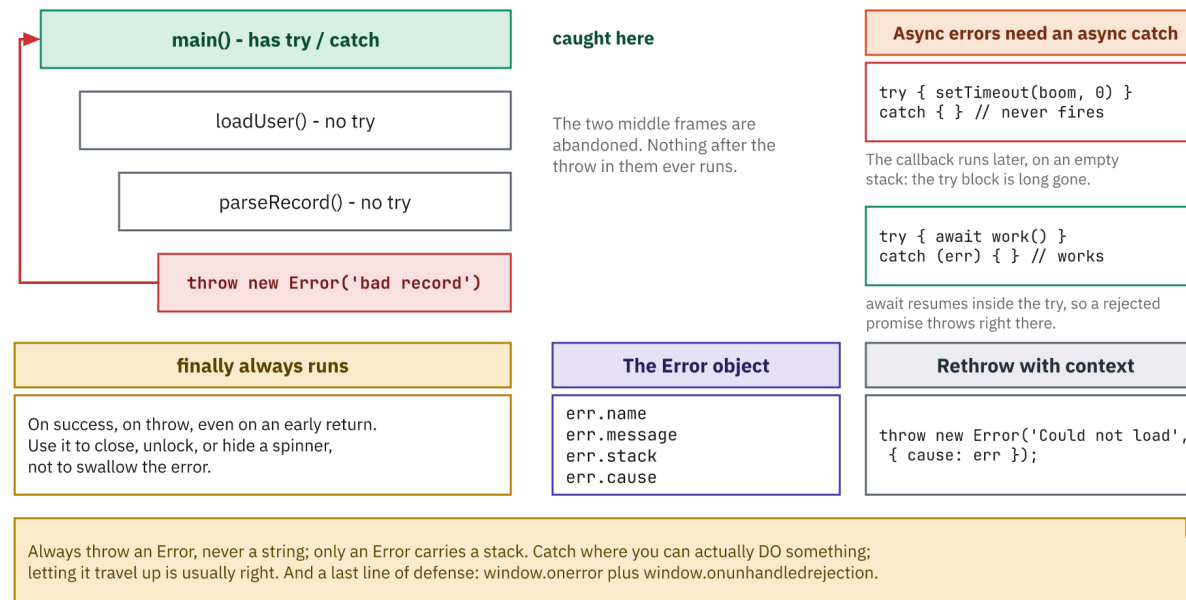
```
async function loadUser() {
  try {
    const res = await fetch("/api/user");
    if (!res.ok) throw new Error("Network error: " + res.status);
    const data = await res.json();
    console.log("User:", data);
  } catch (error) {
    console.error("Could not load user:", error.message);
  }
}
```

- `fetch` only rejects on a **network** failure; check `res.ok` yourself

How a Throw Travels

Error Propagation

A throw unwinds the stack until it meets a catch. Frames in between just disappear.



- It unwinds the stack to the nearest **catch**; frames in between are abandoned
- **finally** always runs; a sync **try** cannot catch an async throw

Callbacks and Timers

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Single-Threaded Execution

- JavaScript runs in **one thread**, one thing at a time
- Each piece of code must **finish** before the next starts
- Currently running code lives on the **call stack**
- Async behavior is what makes this workable

The Host Environment

- The browser or Node.js provides extra features:
 - Timers (timeouts, intervals)
 - Network requests
 - DOM events (click, input, etc.)
- These are **not** part of the JS language; they are **Web/host APIs**
- The environment starts the work, then queues the callback when ready

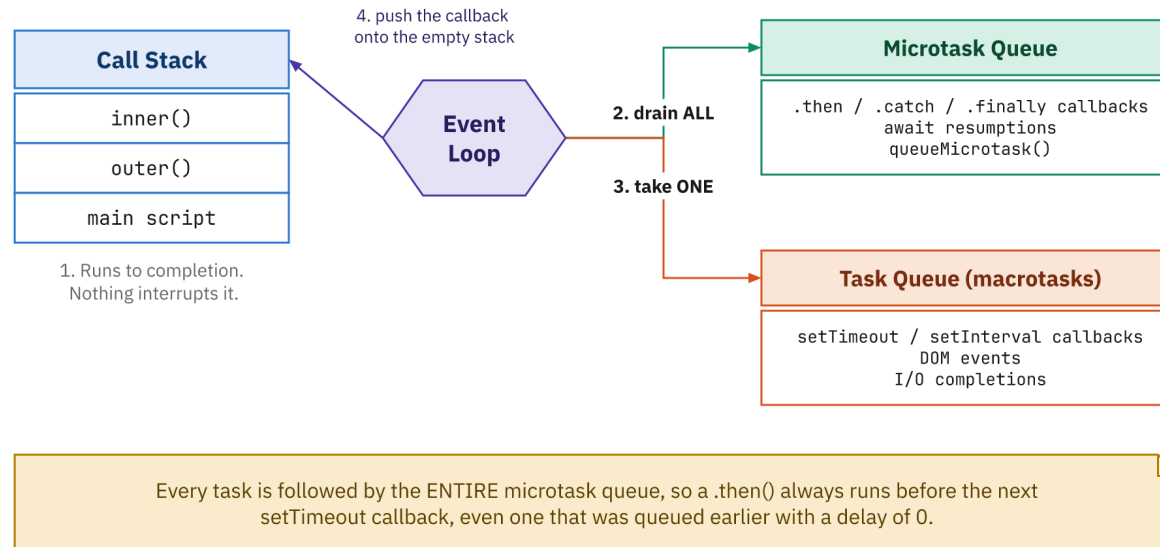
Tasks and Microtasks

- Two main queues feed the event loop:
 - **Task (macrotask) queue**: timers, UI events, network callbacks
 - **Microtask queue**: promises, high-priority small tasks
- Key rule: after each task, run **all microtasks** before the next task
- This is why promise callbacks run **before** timeout callbacks

The Event Loop Cycle

The Event Loop

One thread. The stack must be empty before anything queued can run.



- Stack empties → drain **all** microtasks → take **one** task → repeat
- A `.then()` always beats a `setTimeout`, even one queued earlier

Blocking the Event Loop

```
setTimeout(() => console.log("timer - finally!"), 0);

const stop = Date.now() + 1000;
while (Date.now() < stop) {} // busy for a full second
console.log("blocked for 1000ms");
// blocked for 1000ms
// timer - finally!
```

- While the stack is busy, nothing queued can run
- In a browser the page freezes: no paint, no clicks, no scrolling

What Is a Callback?

- A function you **pass to another function** to be called later
- Core to how JavaScript does asynchronous work
- Can be **synchronous** (runs immediately) or **asynchronous** (runs later)

```
function greet(name) {  
  console.log("Hello, " + name);  
}  
function doTwice(callback) {  
  callback("Alice");  
  callback("Bob");  
}  
doTwice(greet); // synchronous callback
```

Asynchronous Callbacks

- The callback runs **later**, via the event loop

```
console.log("Start");
setTimeout(function () {
  console.log("Inside timeout");
}, 1000);
console.log("End");
// Start
// End
// Inside timeout  <- runs later
```

- `setTimeout` registers the callback and moves on

Callbacks and DOM Events

- Event listeners are callbacks too

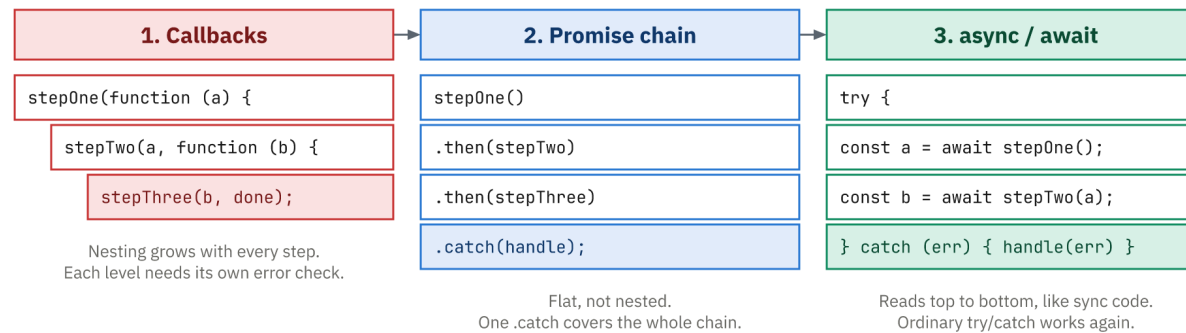
```
button.addEventListener("click", function () {  
  console.log("Button was clicked!");  
});
```

- You register the callback; the browser listens outside the JS engine
- On a click, the browser queues the callback
- The event loop runs it when the call stack is free

Callback Hell

The Async Progression

The same three dependent steps, written three ways. Only the shape of the code changes.



All three do the same work on the same event loop. async/await is not a new mechanism: an async function returns a promise, and every await is a .then() the engine writes for you.

- Nesting grows with every step, and each level needs its own error check
- Motivates **promises** and `async / await`: same event loop, better shape

setTimeout: Run Once, Later

- `setTimeout(callback, delay)` runs a function **once** after at least `delay` ms

```
console.log("start");
setTimeout(() => {
  console.log("later");
}, 1000);
console.log("end");
// start
// end
// later  <- ~1 second later
```

- The delay is a **minimum**, not a guarantee

Arguments and Canceling

- Extra arguments after the delay pass to the callback

```
setTimeout((name) => {  
  console.log("Hello, " + name);  
}, 500, "Alice"); // Hello, Alice
```

- `setTimeout` returns an **id**; cancel with `clearTimeout`

```
const id = setTimeout(() => console.log("never"), 1000);  
clearTimeout(id);
```

setInterval: Run Repeatedly

- `setInterval(callback, delay)` repeats until you stop it

```
let count = 0;
const id = setInterval(() => {
  count++;
  console.log("tick " + count);
  if (count === 3) clearInterval(id);
}, 1000);
// tick 1, tick 2, tick 3 (then stops)
```

- **Always keep the id:** an uncleared interval never lets the program exit

queueMicrotask: Run Very Soon

- Schedules on the **microtask queue**, same one promises use
- Runs after current code, but **before any timer**

```
console.log("1: sync");
setTimeout(() => console.log("2: timeout"), 0);
queueMicrotask(() => console.log("3: microtask"));
console.log("4: sync");
// 1: sync
// 4: sync
// 3: microtask  <- before timers
// 2: timeout
```

Pitfall: Interval Drift

- `setInterval` does **not** guarantee even spacing; ticks drift
- For accuracy, self-correct with a `setTimeout` chain

```
function everySecond(callback) {  
  let expected = Date.now() + 1000;  
  function tick() {  
    const drift = Date.now() - expected;  
    callback();  
    expected += 1000;  
    setTimeout(tick, Math.max(0, 1000 - drift));  
  }  
  setTimeout(tick, 1000);  
}
```

Promises

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

What Is a Promise?

- An object representing a value you'll get **now**, **later**, or **never**
- Three states:
 - `pending`: still working
 - `fulfilled`: finished successfully
 - `rejected`: finished with an error

```
const promise = new Promise((resolve, reject) => {  
  const success = true;  
  if (success) resolve("It worked!");  
  else reject(new Error("Something went wrong"));  
});
```

- A promise settles **once**; later `resolve` / `reject` calls are ignored

Consuming a Promise

- `.then` handles success, `.catch` handles errors

```
promise
  .then((value) => {
    console.log("Success:", value);
  })
  .catch((error) => {
    console.log("Error:", error.message);
  });
```

- `resolve` → fulfilled; `reject` → rejected

Always Reject With an **Error**

```
reject("Failed to load");           // legal, but avoid  
reject(new Error("Failed to load")); // do this
```

- A string rejection has no `.message`, no `.stack`, no `instanceof Error`
- Your `.catch` then has to guess what it was handed
- Same rule as `throw`; see Error Handling

Promises vs Callbacks

- Plain callbacks nest into pyramids; error handling gets messy
- Promises **chain**: each `.then` returns a new promise

```
doSomething()  
  .then((result) => doSomethingElse(result))  
  .then((nextResult) => console.log("Next:", nextResult))  
  .catch((error) => console.error("Error:", error.message));
```

- One central `.catch` handles the whole chain

You Must `return` Inside `.then`

```
step1()
  .then((r) => { step2(r); })      // WRONG - nothing waits for step2
  .then(() => console.log("done")); // runs too early

step1()
  .then((r) => step2(r))           // right - return the promise
  .then(() => console.log("done"));
```

- Returning a promise makes the **next** `.then` wait for it
- Forget the `return` and the chain silently runs out of order

`.finally` and Cleanup

```
showSpinner();

loadData()
  .then((data) => render(data))
  .catch((error) => showError(error.message))
  .finally(() => hideSpinner()); // runs either way
```

- `.finally` gets no value and does not change the result
- The right home for spinners, unlocking, re-enabling a button

Promises and the Event Loop

- `.then` / `.catch` / `.finally` never run immediately
- They are scheduled on the **microtask queue**
- The loop finishes current code, drains **all microtasks**, then the next task
- So promise handlers run **before** `setTimeout(..., 0)`

Ordering Example

```
console.log("A");
Promise.resolve().then(() => console.log("Promise"));
setTimeout(() => console.log("Timeout"), 0);
console.log("B");
// A
// B
// Promise    <- microtask
// Timeout    <- macrotask
```

- Sync code first, then microtasks, then timers

Converting Callbacks to Promises

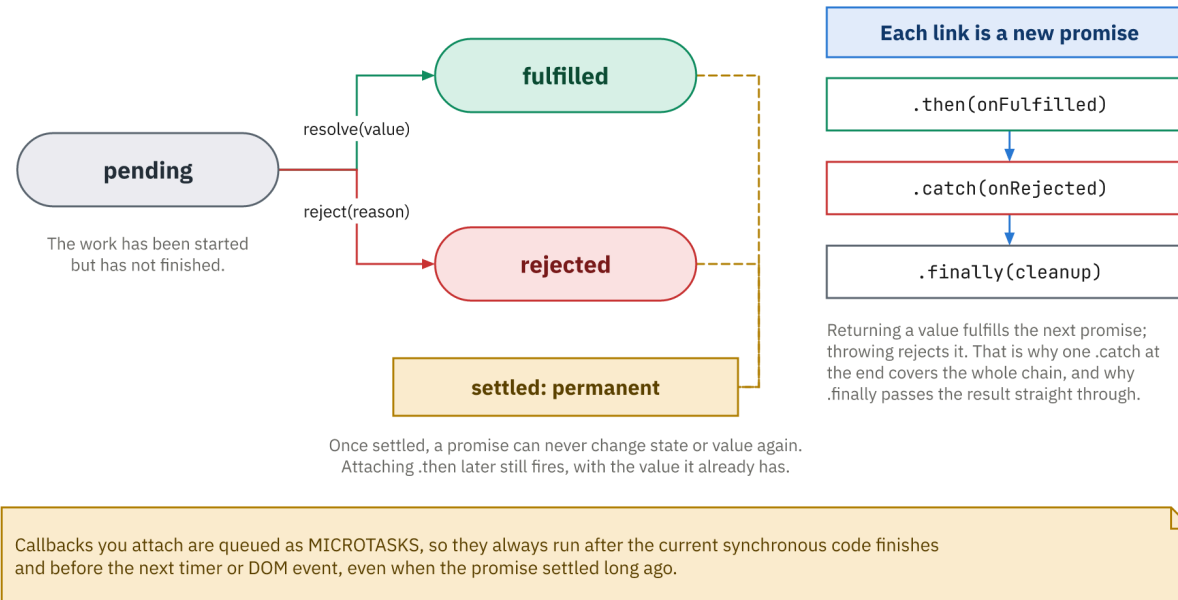
```
function loadData() {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      resolve("Data loaded");  
    }, 1000);  
  });  
}  
  
loadData()  
  .then((data) => console.log(data))  
  .catch((error) => console.error(error.message));
```

- Now works with `.then / .catch`, `Promise.all`, and `async/await`

States and the Chain

Promise States

A promise settles once and stays settled. Every `.then` returns a NEW promise.



- Settled is permanent, and every `.then` returns a **new** promise
- Callbacks are queued as microtasks, so they never run synchronously

async / await

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Async Code That Reads Like Sync

- `async/await` makes async code look step-by-step
- Built on **promises**; still uses the **event loop**

```
async function showUserAndPosts() {  
  try {  
    const user = await loadUser();  
    const posts = await loadPosts(user.id);  
    console.log("User:", user);  
    console.log("Posts:", posts);  
  } catch (error) {  
    console.error(error.message);  
  }  
}  
showUserAndPosts();
```

How It Works

- Two key rules:
 1. An `async` function **always returns a promise**
 2. `await` pauses until that promise settles

```
async function getNumber() {  
  return 42; // actually Promise.resolve(42)  
}  
  
getNumber();           // Promise { 42 } - not 42!  
await getNumber();     // 42
```

- `await` returns the fulfilled value, or **throws** the rejection

The Same Chain, Both Ways

```
// promises
step1()
  .then((a) => step2(a))
  .then((b) => step3(b))
  .catch((e) => console.error(e.message));

// async/await - identical behavior
try {
  const a = await step1();
  const b = await step2(a);
  await step3(b);
} catch (e) {
  console.error(e.message);
}
```

Where `await` Is Allowed

- Inside any `async` function
- At the **top level of an ES module** (top-level `await`, ES2022)

```
// app.mjs, or a .js file in a "type": "module" package
const res = await fetch("/api/config");
const config = await res.json();
```

- In a plain CommonJS script: `SyntaxError: await is only valid in async functions`
- When in doubt, wrap it in an `async` function; that works everywhere

Error Handling with try/catch

- Any rejected promise in the `try` block acts like a thrown error

```
async function run() {  
  try {  
    const data = await loadData();  
    const result = await processData(data);  
    console.log("Result:", result);  
  } catch (error) {  
    console.error("Error:", error.message);  
  }  
}  
run();
```

- One `catch` can handle them all

Fine-Grained Error Handling

- Use multiple `try` / `catch` blocks for separate control

```
async function run() {  
  let data;  
  try {  
    data = await loadData();  
  } catch (error) {  
    console.error("Failed to load data:", error.message);  
    return;  
  }  
  try {  
    const result = await processData(data);  
    console.log("Result:", result);  
  } catch (error) {  
    console.error("Failed to process data:", error.message);  
  }  
}
```

Async/Await and the Event Loop

- At each `await`:
 - The `async` function **pauses**
 - The rest is scheduled as a **microtask** (like a `.then`)
 - The event loop can run other tasks meanwhile
 - When the promise settles, the function resumes
- In short: nice syntax for promises, same microtask queue

The Bug Everyone Writes Once

```
async function getTotal() { return 42; }

const total = getTotal();           // Promise { 42 }
console.log(total + 1);             // "[object Promise]1"

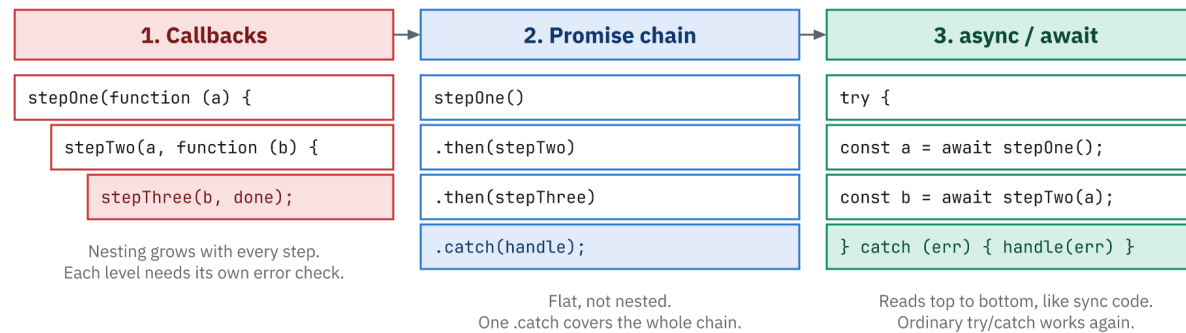
const real = await getTotal();      // 42
console.log(real + 1);              // 43
```

- Forgetting `await` rarely errors; it just gives you a promise
- If a value prints as `Promise { ... }`, you missed an `await`

The Same Three Steps, Three Ways

The Async Progression

The same three dependent steps, written three ways. Only the shape of the code changes.



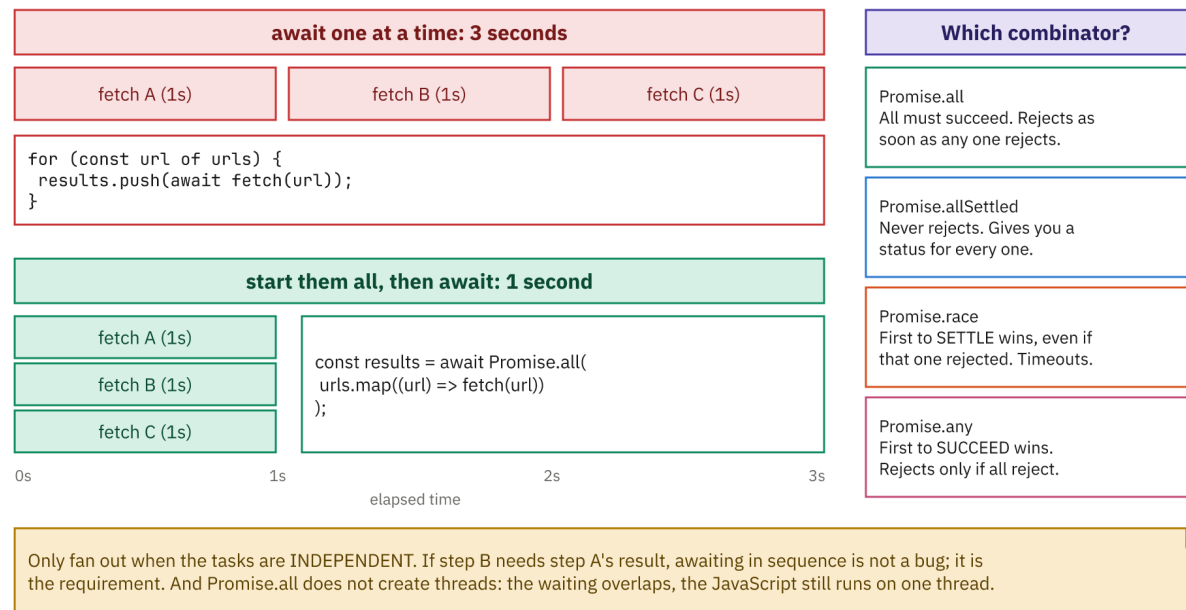
All three do the same work on the same event loop. async/await is not a new mechanism: an async function returns a promise, and every await is a .then() the engine writes for you.

- `async / await` is not a new mechanism; every `await` is a `.then`
- Ordinary `try / catch` works again

Sequential vs Concurrent

Sequential vs Concurrent

Awaiting in a loop is a queue. `Promise.all` is a starting gun for tasks that do not depend on each other.



- Awaiting in a loop is a queue; `Promise.all` is a starting gun
- Only fan out when the tasks are genuinely independent

Concurrent Promises

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Sequential Costs the Sum

- Awaiting one, then the next, runs them **one at a time**

```
const a = await fetchUser(1); // 1s
const b = await fetchUser(2); // + 1s
const c = await fetchUser(3); // + 1s
// ~3 seconds total
```

- Correct when each step **needs** the previous result
- Wasteful when the three jobs are independent

Concurrent Costs the Slowest

- Start them all, then wait for the group

```
const [a, b, c] = await Promise.all([
  fetchUser(1),
  fetchUser(2),
  fetchUser(3),
]);
// ~1 second total
```

- The promises start the moment they are **created**, not when awaited
- `Promise.all` just waits for the whole array

Results Come Back in Order

```
const [slow, fast] = await Promise.all([
  delay(300, "slow"),
  delay(50, "fast"),
]);

console.log(slow, fast); // "slow" "fast"
```

- The array order matches the order you **passed in**
- Not the order they finished, so destructuring is always safe

Promise.all Is All-or-Nothing

```
try {
  const results = await Promise.all([
    ok("a"),
    Promise.reject(new Error("boom")),
    ok("c"),
  ]);
} catch (error) {
  console.log(error.message); // "boom"
}
```

- **One rejection rejects the whole thing**, immediately
- You lose the results that *did* succeed
- The other promises are not canceled; they just finish unnoticed

Promise.allSettled Never Short-Circuits

```
const results = await Promise.allSettled([
  ok("a"),
  Promise.reject(new Error("boom")),
]);

console.log(results);
// [ { status: "fulfilled", value: "a" },
//   { status: "rejected", reason: Error: boom } ]
```

- Always fulfills, with one outcome object per promise
- Use it when partial success is still useful, a dashboard of widgets

Reading `allSettled` Results

```
const ok = results.filter((r) => r.status === "fulfilled");
const bad = results.filter((r) => r.status === "rejected");

console.log(`${ok.length} of ${results.length} loaded`);

const values = ok.map((r) => r.value);
const reasons = bad.map((r) => r.reason.message);
```

- Fulfilled entries carry `value`; rejected ones carry `reason`

Promise.race and Promise.any

```
// race - first to SETTLE wins, success or failure
const winner = await Promise.race([slowFetch(), timeout(5000)]);

// any - first to SUCCEED wins; ignores rejections
const fastest = await Promise.any([mirror1(), mirror2(), mirror3()]);
```

- `race` is the timeout pattern: whichever finishes first
- `any` rejects only if **every** promise rejects (an `AggregateError`)

Choosing the Right One

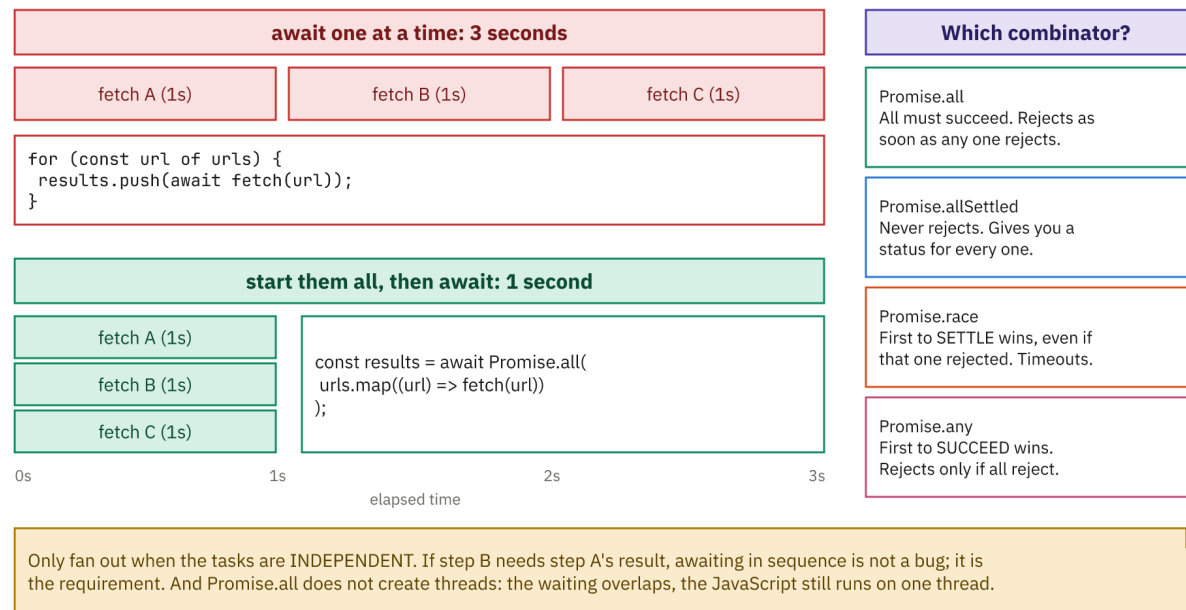
You need	Use
All of them, fail fast	<code>Promise.all</code>
Every outcome, good and bad	<code>Promise.allSettled</code>
First to finish, either way	<code>Promise.race</code>
First success, ignore failures	<code>Promise.any</code>
Each step needs the last	plain sequential <code>await</code>

- Reach for `all` by default; `allSettled` when a failure is survivable

Choosing a Combinator

Sequential vs Concurrent

Awaiting in a loop is a queue. `Promise.all` is a starting gun for tasks that do not depend on each other.



- `all` needs every one; `allSettled` never rejects
- `race` takes the first to settle; `any` the first to succeed

JavaScript Modules

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Why Modules?

- Split code into separate files, each with its own **scope**
- **Export** things from one file, **import** them in another
- Easier to organize, reuse, and reason about

```
// math.js
export function add(a, b) { return a + b; }

// main.js
import { add } from "./math.js";
```

Before Modules: Multiple Scripts

- The old way to split browser code across files:

```
<script src="helpers.js"></script>  
<script src="app.js"></script>
```

- Run in **document order**: `app.js` must come after what it uses
- All share **one global scope** (`window`), easy to **collide**
- Modules fix this: per-file scope + explicit `import` / `export`

Named Exports

- Many per module, imported by exact name
- Inline with the declaration, or as a list

```
export const PI = 3.14159;  
export function add(a, b) {  
  return a + b;  
}  
  
const x = 1, y = 2;  
function multiply(a, b) { return a * b; }  
export { x, y, multiply };
```

- Rename on export: `export { secret as answer };`

Default Export

- Each module may have **one** default export

```
export default function greet(name) {  
  console.log("Hello, " + name);  
}
```

- Can also default-export a value or class

```
const config = { debug: true };  
export default config;  
// export default class Animal {}
```

Importing Named and Default

```
import { PI, add } from "./math.js";           // named
import { add as sum } from "./math.js";        // renamed
import greet from "./greet.js";                // default
import greet, { PI, add } from "./mod.js";     // both
```

- Named imports need the exact exported names in braces
- The default import name is your choice
- The braces are **not** destructuring; it is import syntax

Namespace and Side-Effect Imports

- **Namespace:** collect all named exports into one object

```
import * as math from "./math.js";  
// math.PI, math.add(...) (default is math.default)
```

- **Side-effect only:** run a module for its effects, no bindings

```
import "./setup.js";
```

Re-Exporting

- Export bindings straight from another module

```
export { add, PI } from "./math.js";  
export { add as sum } from "./math.js";  
export * from "./math.js";          // all named exports
```

- Mix local and re-exported, the "barrel file" pattern

```
export const version = "1.0.0";  
export * from "./math.js";
```

Top-Level Only

- Static `import` / `export` must be at the **top level**
- Not allowed inside functions, `if`, loops, etc.

```
// valid
import { add } from "./math.js";

// invalid
if (true) {
  import { add } from "./math.js";
}
```

- Structure is known before code runs, that's what makes them **static**

Imports Are Hoisted

```
console.log(add(2, 3)); // 5 - works!  
import { add } from "./math.js";
```

- The whole module graph is resolved **before** any code runs
- So import position does not matter, but put them at the top anyway
- This is also why the path must be a literal string, not built at runtime

In the Browser: `type="module"`

- Mark a `<script>` as a module to use `import` / `export`

```
<script type="module" src="main.js"></script>
```

```
<script type="module">  
  import { add } from "./math.js";  
  console.log(add(5, 7));  
</script>
```

- Module scripts are **deferred** automatically (run after HTML parsing)
- The `.js` extension in the path is **required**

In Node: Marking a File as an ES Module

- Node supports both ES modules and CommonJS (`require`)
- A file is an ES module if:
 - The nearest `package.json` has `"type": "module"`, or
 - The file uses the `.mjs` extension
- Once it's an ES module, the syntax matches the browser
- A `.cjs` file stays CommonJS even inside a `"type": "module"` package

CommonJS: The Older System

```
// utils.cjs
exports.add = function (a, b) { return a + b; };
// or: module.exports = { add };

// index.cjs
const utils = require("./utils.cjs");
console.log(utils.add(2, 3));
```

- `require` is **synchronous** and resolved at **runtime**
- Still everywhere in older Node code; you will read it

Reaching CommonJS From an ES Module

- There is no `require` in an ES module, make one

```
import { createRequire } from "node:module";
const require = createRequire(import.meta.url);

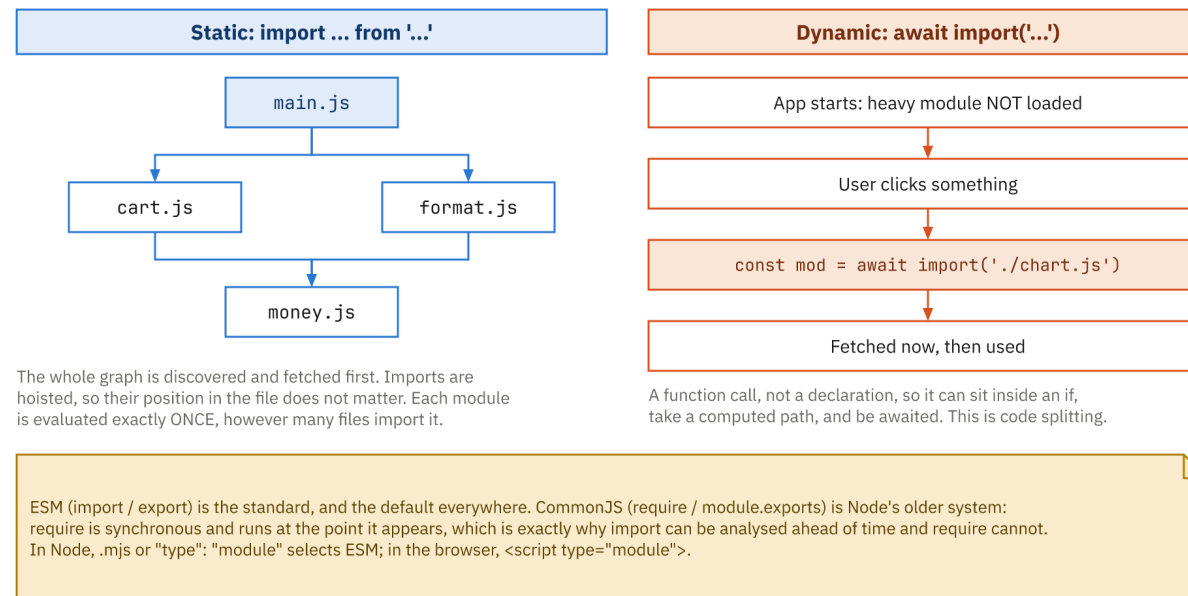
const legacy = require("./legacy-utils.cjs");
console.log(legacy.add(2, 3));
```

	ES modules	CommonJS
Syntax	<code>import</code> / <code>export</code>	<code>require</code> / <code>module.exports</code>
Resolved	before run (static)	at runtime
Top-level <code>await</code>	yes	no

The Module Graph

The Module Graph

Static imports are resolved before any of your code runs. Dynamic `import()` happens later, and returns a promise.



- `import` is resolved before any of your code runs
- Each module is evaluated exactly **once**, however many files import it

The window, the location, and Selecting Elements

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

What is `window`?

- The **global object** for browser JavaScript
- Represents the browser **tab**
- Holds `document`, `location`, `history`, `localStorage`
- Provides `setTimeout`, `alert`, `addEventListener`, and more

```
window === this;      // true (global script)
window === globalThis; // true (browser)
```

Globals, Size, and Scroll

- `var` globals become properties of `window`; `let` / `const` do not
- Prefer `const` / `let` and modules; don't pollute the global scope

```
var greeting = "hi";
window.greeting;    // "hi"

window.innerWidth;  // viewport width in px
window.innerHeight; // viewport height in px
window.scrollTo;    // vertical scroll offset

window.scrollTo({ top: 500, behavior: "smooth" });
```

Other **window** Members

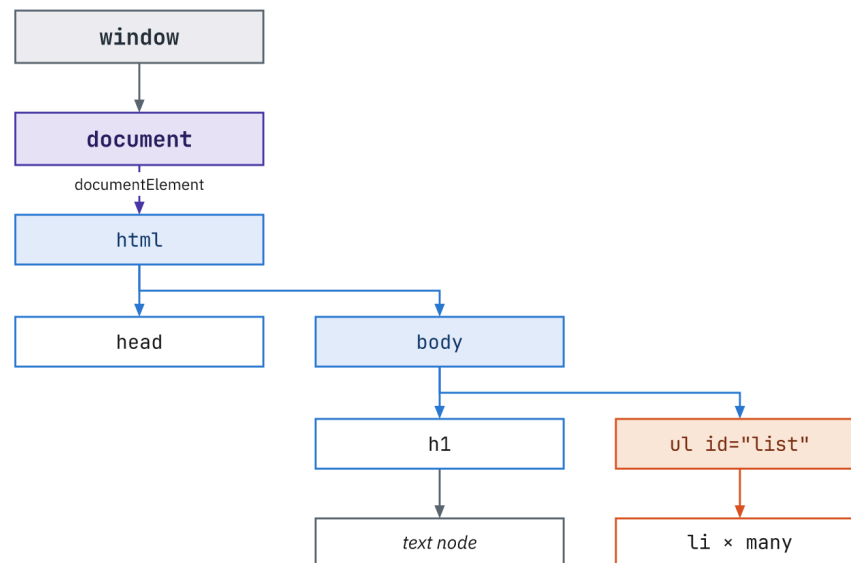
- **navigator**: browser and environment info
- **localStorage** / **sessionStorage**: string key/value storage
- **alert**, **confirm**, **prompt**: simple blocking dialogs

```
localStorage.setItem("theme", "dark");  
localStorage.getItem("theme");    // "dark"  
localStorage.getItem("missing"); // null  
  
console.log(navigator.onLine);    // true / false
```


What is **document** ?

The DOM Tree

The browser parses your HTML into a tree of objects. JavaScript changes the tree, not the file.



Reaching into the tree

```
getElementById('list')
querySelector('#list li')
querySelectorAll('li')
closest('ul')
```

`querySelectorAll` returns a static `NodeList`: a snapshot, not live.

window is the global object

```
window.document
window.location
window.navigator
window.localStorage
```

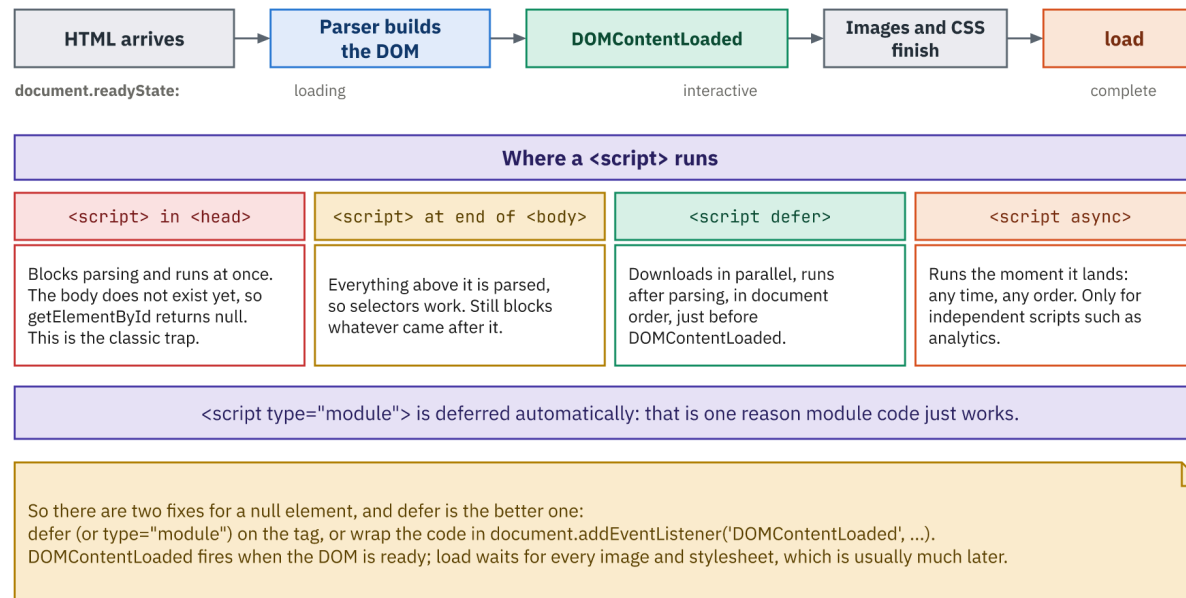
Everything global hangs off it, so `window` is usually optional.

- The **entry point to the page content**: **documentElement**, **head**, **body**
- JavaScript changes the tree in memory, never the HTML file

Running Code When Ready

Page and Script Lifecycle

Why the element was null: your script ran before the browser had parsed it.



- A `<script>` in `<head>` runs before `<body>` exists, hence the `null`
- Fix it with `defer`, `type="module"`, or a `DOMContentLoaded` listener

Reading the URL

- `location` (same as `window.location`) describes the current URL

```
// https://example.com:8080/shop?category=books#reviews
location.href;      // full URL
location.protocol;  // "https:"
location.host;      // "example.com:8080"
location.pathname;  // "/shop"
location.search;    // "?category=books"
location.hash;      // "#reviews"
location.origin;    // "https://example.com:8080"
```

Query Parameters

- `location.search` holds the raw query string; parse it, don't split it

```
const params = new URLSearchParams(location.search);

params.get("category"); // "books"
params.get("missing");  // null
params.has("page");     // false
params.getAll("tag");    // every value for a repeated key

params.forEach((v, k) => console.log(k, "=", v));
```

Navigating

- `href` / `assign`: go to a URL (Back returns)
- `replace`: swap the current entry (Back skips it)
- `reload`: reload the current page, **no arguments**

```
location.href = "https://example.com/profile";  
location.assign("https://example.com/profile");  
location.replace("https://example.com/login"); // no back  
location.reload();
```

querySelector

- Returns the **first** element matching a CSS selector
- Works with `#id`, `.class`, `tag`, or any CSS selector

```
document.querySelector("#main-title");  
document.querySelector(".highlight");  
document.querySelector("button");  
document.querySelector(".menu .item.active");
```

- Returns `null` when nothing matches; guard before using it

querySelectorAll

- Returns **all** matches as a static NodeList
- Has `forEach`; convert with `Array.from` for `map` / `filter`

```
const todos = document.querySelectorAll(".todo");
console.log(todos.length);

todos.forEach((item, i) => console.log(i, item.textContent));

const arr = Array.from(todos).map((el) => el.textContent);
```

Older Selection Methods

- `getElementById`: one element by id (no `#`)
- `getElementsByClassName` / `getElementsByTagName`: **live** collections

```
document.getElementById("user-card");  
document.getElementsByClassName("note"); // live HTMLCollection  
document.getElementsByTagName("div");    // live HTMLCollection
```

- **Live** means the collection updates itself as the DOM changes
- `querySelectorAll` gives a **static** snapshot, usually what you want

Creating and Inserting

- `createElement` builds a node; `textContent` fills it; then append

```
const list = document.getElementById("list");
const frag = document.createDocumentFragment();

for (let i = 1; i <= 5; i++) {
  const li = document.createElement("li");
  li.textContent = "Item " + i;
  frag.appendChild(li);
}
list.appendChild(frag); // one append, not five
```

Geolocation: Ask, Don't Assume

- Lives on `navigator.geolocation`, another `window` API
- Needs **user permission** and a **secure context** (https or `localhost`)
- It cannot return a value, because the browser has to prompt first

```
if ("geolocation" in navigator) {  
    console.log("Geolocation is supported");  
}
```

- So it takes **callbacks**, the first async API in the course

Geolocation: `getCurrentPosition`

```
navigator.geolocation.getCurrentPosition(  
  (position) => {  
    const { latitude, longitude, accuracy } = position.coords;  
    console.log(latitude, longitude, accuracy);  
  },  
  (error) => {  
    // A declined prompt is EXPECTED - warn, don't error  
    console.warn("Unavailable:", error.message);  
  },  
  { timeout: 10000, enableHighAccuracy: false },  
);
```

- The error callback is **not** optional; users refuse

DOM Events, Delegation, and Styling

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Scoped, `matches`, `closest`

- Query **inside** an element, not just `document`
- `matches`: does this element match a selector?
- `closest`: nearest matching ancestor (including itself)

```
const card = document.querySelector(".card");  
const title = card.querySelector(".title"); // search inside  
  
el.matches(".item.active"); // true / false  
button.closest(".menu"); // nearest ancestor <ul>
```

- `closest` is what makes event delegation reliable

Modifying Content

- `textContent`: plain text (safe)
- `innerHTML`: parses HTML (never with untrusted input)

```
title.textContent = "New title";  
  
box.innerHTML = "<strong>Bold</strong> and <em>italic</em>";  
// Avoid innerHTML with user input - XSS risk
```

- If the value came from a user or a server, use `textContent`

Attributes and Classes

- Set attributes directly or with `setAttribute`
- Prefer `classList` over replacing `className`

```
img.src = "new.png";  
img.setAttribute("data-role", "avatar");  
img.getAttribute("data-role"); // "avatar"  
  
panel.classList.add("active");  
panel.classList.remove("active");  
panel.classList.toggle("active");  
panel.classList.contains("active"); // true / false
```

dataset: Attach Data to an Element

- Any `data-*` attribute is readable through `dataset`

```
<li data-id="42" data-user-name="ada">Ada</li>
```

```
li.dataset.id;          // "42"    - always a STRING  
li.dataset.userName;    // "ada"    - data-user-name becomes userName  
  
li.dataset.id = 43;     // writes data-id="43"  
Number(li.dataset.id);  // convert when you need a number
```

- The natural place to keep the record id a list item represents

Removing Elements

- `element.remove()`: modern, removes itself
- `parent.removeChild(child)`: older style
- `innerHTML = ''` clears all children

```
button.remove();  
  
list.removeChild(item);  
  
log.innerHTML = ""; // clear everything inside
```

Adding and Removing Listeners

- `addEventListener(type, handler, options?)`
- Remove with the **same** function reference
- Anonymous functions cannot be removed

```
function onClick() { console.log("clicked"); }  
  
btn.addEventListener("click", onClick);  
btn.removeEventListener("click", onClick);
```

- Listen on the **form** for `submit`, not `click` on the button

The Event Object

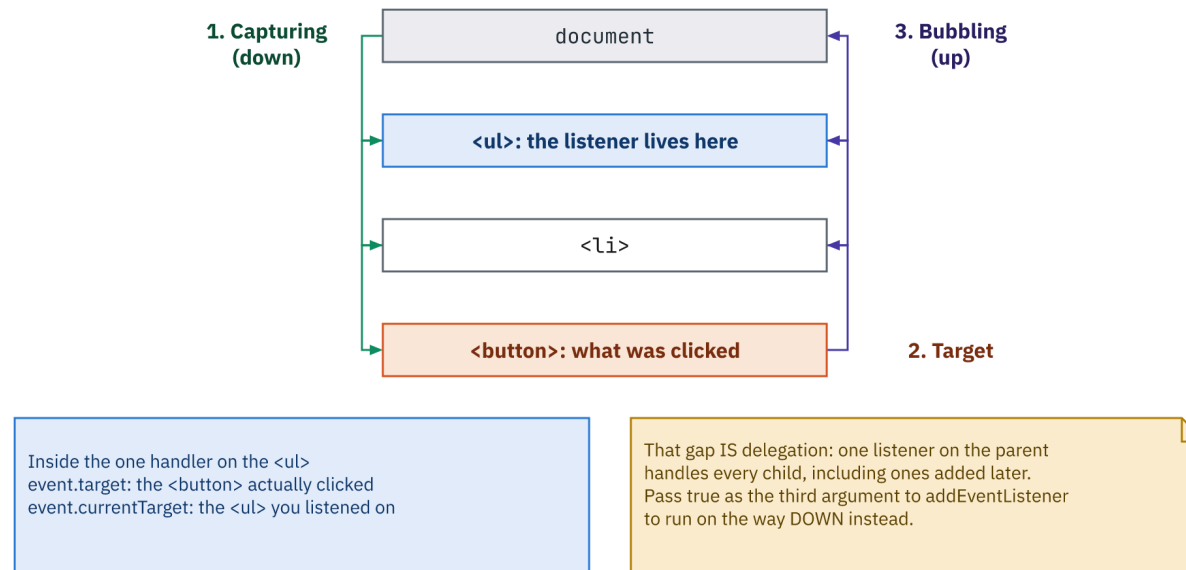
- Handlers receive an event with details
- `type`, `target`, mouse/key data

```
btn.addEventListener("click", (event) => {  
  console.log(event.type);    // "click"  
  console.log(event.target);  // the clicked element  
  console.log(event.clientX); // mouse X  
});
```

Capture and Bubble

The DOM Event Path and Delegation

One click travels down to the deepest element, then back up. Listening high up is delegation.



- Listeners default to **bubbling**; pass `{ capture: true }` for the way down
- `target` is what was clicked, `currentTarget` is where you listened

Stopping and Preventing

- `stopPropagation()`: halt the event's travel
- `preventDefault()`: cancel the browser default
- `{ once: true }`: auto-remove after the first run

```
innerBtn.addEventListener("click", (e) => {  
  e.stopPropagation(); // outer won't be notified  
});  
  
link.addEventListener("click", (e) => {  
  e.preventDefault(); // don't navigate  
});
```

Event Delegation

- One parent listener handles many children
- Works for elements added **later**

```
list.addEventListener("click", (event) => {  
  const item = event.target.closest(".item");  
  if (!item || !list.contains(item)) return;  
  
  console.log("Clicked:", item.textContent, item.dataset.id);  
});
```

- The gap between `target` and `currentTarget` is what makes this work

Inline Styles

- `element.style` sets inline styles (highest priority)
- Properties are **camelCase**; values are strings with units

```
btn.style.color = "white";  
btn.style.backgroundColor = "blue"; // not "background-color"  
btn.style.padding = "10px 20px";  
  
btn.style.width = 50;    // ignored - no unit  
btn.style.width = "50px"; // works
```

Styling with Classes

- Preferred: keep styles in CSS, toggle classes in JS

```
message.classList.add("highlight");  
message.classList.remove("highlight");  
message.classList.toggle("highlight");  
  
if (message.classList.contains("highlight")) {  
    console.log("It is highlighted");  
}
```

- One class swap can drive a dozen CSS rules, and animate

CSS Variables and Computed Styles

- Set CSS custom properties as theme "knobs"
- `getComputedStyle` reads the **final** applied values (read-only)

```
document.documentElement.style
  .setProperty("--main-bg", "black");

const styles = getComputedStyle(box);
console.log(styles.color);          // "rgb(0, 0, 255)"
console.log(styles.paddingTop);    // "20px"
```

- `element.style.color` shows only **inline** styles, often empty

Choosing an Approach

- **Classes:** reusable styles, CSS transitions, maintainability
- **Inline:** a single dynamic value computed in JS
- **CSS variables:** theme/settings shared across many rules

```
// dynamic value -> inline style is natural  
bar.style.width = value + "%";
```

The Fetch API

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Basic Idea

- Modern way to make HTTP requests from the browser
- Built on **promises**: works with `.then()` and `async/await`
- `fetch(url, options?)` resolves to a `Response`

```
const res = await fetch("https://api.example.com/users");
const data = await res.json();
console.log(data);
```

- Two steps: the **response arrives**, then you **read the body**

fetch Does Not Throw on 404

```
const res = await fetch("/api/missing");

console.log(res.ok);      // false
console.log(res.status); // 404
// ...but no error was thrown!
```

- A promise rejection means a **network** failure, not an HTTP error
- The server answering "404" is a successful round trip, as far as `fetch` is concerned
- Always check `res.ok` yourself

GET with async/await

- Check `response.ok` before parsing
- Network and thrown errors land in `catch`

```
async function getUsers() {  
  try {  
    const res = await fetch("https://api.example.com/users");  
    if (!res.ok) throw new Error("Status " + res.status);  
  
    const data = await res.json();  
    console.log("Users:", data);  
  } catch (error) {  
    console.error("Fetch error:", error.message);  
  }  
}
```

POST: Create a Resource

- Set `method` and a `Content-Type` header
- `body` must be a string; use `JSON.stringify`

```
const res = await fetch("https://api.example.com/users", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ name: "Alice", email: "a@example.com" }),
});

if (!res.ok) throw new Error("Failed to create user");
const created = await res.json();
```

PUT, PATCH, DELETE

- **PUT**: replace the whole resource
- **PATCH**: update only some fields
- **DELETE**: remove; may return **204 No Content**

```
// PATCH: partial update
await fetch(`https://api.example.com/users/${id}`, {
  method: "PATCH",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ email }),
});

// DELETE
await fetch(`https://api.example.com/users/${id}`, {
  method: "DELETE",
});
```


Reading the Response

```
await res.json(); // parse JSON -> object
await res.text(); // raw text -> string
await res.blob(); // binary -> Blob

res.headers.get("content-type"); // "application/json"
```

- You can read the body **once**; a second `.json()` throws
- A `204 No Content` has no body at all, so do not call `.json()` on it

Canceling a Request: **AbortController**

- A request with no timeout can hang forever

```
const controller = new AbortController();
const timer = setTimeout(() => controller.abort(), 5000);

try {
  const res = await fetch(url, { signal: controller.signal });
  clearTimeout(timer);
} catch (error) {
  if (error.name === "AbortError") console.log("Canceled");
  else throw error;
}
```

- Also use it when the user navigates away or retypes a search

Fetch and the Event Loop

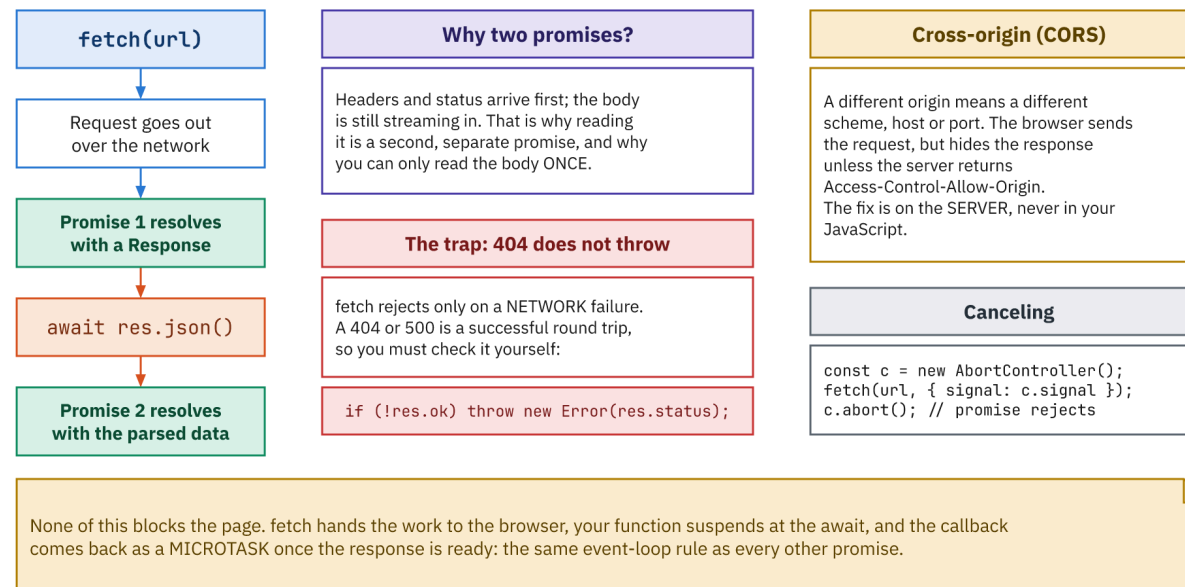
- The request runs in the browser's **Web APIs**, not in JS itself
- The response resolves the promise
- The continuation is scheduled as a **microtask**
- The event loop runs it when the stack is free

"Start a network request now, run this code later when the result is ready."

Two Promises, Not One

The Fetch Lifecycle

Two promises, not one: the first gives you the response, the second gives you the body.



- The body is a stream, so `.json()` is a second promise
- A 404 does not throw; check `res.ok` yourself

Strings, Numbers and Math

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Length, Indexing, Immutability

```
let coffee = "espresso";  
coffee.length;    // 8  
coffee[0];        // "e"  
coffee.at(-1);    // "o" - negative index
```

- Strings are **immutable**: you cannot change a character in place

```
let word = "hello";  
word[0] = "H";    // silently ignored  
console.log(word); // "hello" (unchanged)
```

- Every string method returns a **new** string

Extracting Substrings

- `slice(start, end)`: end index is **exclusive**, supports negatives

```
let text = "Summit Coffee";  
text.slice(0, 6);    // "Summit"  
text.slice(7);       // "Coffee"  
text.slice(-6);      // "Coffee" (last 6 chars)
```

- `substring` is similar but ignores negatives; prefer `slice` in new code

Searching a String

```
let title = "The Summit Coffee Roasters";
title.indexOf("Coffee");    // 11
title.indexOf("Tea");       // -1 (not found)
title.includes("Coffee");   // true
title.startsWith("The");    // true
title.endsWith("Roasters"); // true
```

- All are **case-sensitive**; lowercase both sides for insensitive search

```
title.toLowerCase().includes("coffee"); // true
```


Case, Trim, and Padding

```
"Latte".toUpperCase(); // "LATTE"  
"Latte".toLowerCase(); // "latte"  
  
"  alice@example.com  ".trim(); // "alice@example.com"  
  
"5".padStart(3, "0"); // "005"  
"7".padEnd(3, "."); // "7.."  
`00:${String(9).padStart(2, "0")}`; // "00:09"
```

- `trim` is essential when handling form input

Split, Join, Replace, Repeat

```
"espresso,latte,mocha".split(",");  
// ["espresso", "latte", "mocha"]  
  
["a", "b", "c"].join(" | "); // "a | b | c"  
  
"a cat and a cat".replaceAll("cat", "dog");  
// "a dog and a dog"  
  
"=".repeat(10); // "=========="
```

- Prefer `replaceAll` over `replace` (which swaps only the first match)

Number Literals

- One `number` type for both integers and decimals

```
let count = 42;           // integer
let price = 4.5;          // decimal
let big = 1_000_000;      // _ separators (cosmetic)
let hex = 0xff;           // 255
let binary = 0b1010;      // 10
let scientific = 1.5e3;    // 1500
```

Parsing Text Into Numbers

- Form/URL/file input arrives as **strings**; convert it

```
Number("42");    // 42
Number("42px");   // NaN (strict: whole string must be valid)

Number.parseInt("42px", 10); // 42 (lenient: stops at "p")
Number.parseInt("3.9", 10);  // 3  (truncates)
Number.parseFloat("4.5rem"); // 4.5
```

- Always pass a **radix** (usually `10`) to `parseInt`

NaN: Not-a-Number

- Produced when a numeric operation fails
- Infamously **not equal to itself**

```
Number("hello"); // NaN
NaN === NaN;      // false (!)
```

- So test with `Number.isNaN`, not `===`

```
let result = Number("hello");
Number.isNaN(result); // true
```

- Prefer `Number.isNaN` over the misleading global `isNaN()`

Formatting Numbers

- `toFixed(n)` rounds to `n` decimals and returns a **string**

```
(4.5).toFixed(2);    // "4.50"  (a string!)  
(3.14159).toFixed(2); // "3.14"
```

- Do not do math on the result; format only at display time
- `Intl.NumberFormat` for currency and separators

```
new Intl.NumberFormat("en-US", {  
  style: "currency", currency: "USD",  
}).format(1234.5); // "$1,234.50"
```

Floating Point and Radix

```
0.1 + 0.2;           // 0.30000000000000004  
0.1 + 0.2 === 0.3; // false
```

- Not a JS bug; it is IEEE 754, the same in every language

```
(0.1 + 0.2).toFixed(2);           // "0.30" (round to display)  
Math.abs(0.1 + 0.2 - 0.3) < Number.EPSILON; // true (compare with tolerance)  
  
(255).toString(16); // "ff"  
(255).toString(2);  // "11111111"
```

- For money, store integer **cents** and divide by 100 to display

Rounding

- `Math` is a **static namespace**; never `new Math()`

```
Math.round(4.5); // 5    (nearest; .5 rounds up)
Math.floor(4.9); // 4    (down toward -Infinity)
Math.ceil(4.1);  // 5    (up toward +Infinity)
Math.trunc(4.9); // 4    (drop the decimal)
```

- `floor` vs `trunc` differ on **negatives**

```
Math.floor(-4.1); // -5  (down = more negative)
Math.trunc(-4.1); // -4  (chops, stays near zero)
```


Abs, Min, Max, Clamp

```
Math.abs(-7);           // 7 (distance from zero)
Math.max(3, 7, 2, 9, 1); // 9
Math.min(3, 7, 2, 9, 1); // 1

let temps = [61, 72, 58, 80];
Math.max(...temps); // 80 (spread an array)

// clamp volume into 0..100:
Math.min(100, Math.max(0, 130)); // 100
```

Powers and Roots

```
Math.pow(2, 10); // 1024
Math.sqrt(144);  // 12
Math.cbrt(27);   // 3
Math.hypot(3, 4); // 5 (straight-line distance)

2 ** 10;         // 1024 (** operator, often cleaner)
Math.sign(-5);   // -1
Math.PI;         // 3.141592653589793
```

Random Numbers

- `Math.random()` returns a float in `[0, 1)`, never quite 1
- The integer helper is the pattern to memorize (both ends inclusive)

```
function randomInt(min, max) {  
  return Math.floor(Math.random() * (max - min + 1)) + min;  
}  
  
randomInt(1, 6); // a dice roll: 1..6  
  
function pickRandom(array) {  
  return array[randomInt(0, array.length - 1)];  
}
```

- **Not secure**; use `crypto` APIs for tokens or passwords

Forms: Validation and FormData

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

A Form to Work With

- Every field needs a **name**; that is the key JS and the server use

```
<form id="signup" novalidate>
  <input type="text" name="username">
  <input type="email" name="email">
  <input type="number" name="age">
  <label><input type="checkbox" name="newsletter"> Newsletter</label>
  <label><input type="radio" name="plan" value="free" checked> Free</label>
  <label><input type="radio" name="plan" value="pro"> Pro</label>
  <select name="country">
    <option value="us">United States</option>
    <option value="uk">United Kingdom</option>
  </select>
  <button type="submit">Create account</button>
</form>
```

Reaching the Form and Fields

- Grab the form, then reach fields by `name` via its `elements` collection

```
const form = document.getElementById("signup");

const username = form.elements["username"];
const email = form.elements["email"];

console.log(username.name);    // "username"
console.log(username.tagName); // "INPUT"
```

- Cleaner than a `querySelector` for every input

Reading Field Values

```
// Text / email / password / textarea - always a STRING
form.elements["username"].value.trim();
Number(form.elements["age"].value);      // "42" -> 42

// Checkbox: use .checked, NOT .value
form.elements["newsletter"].checked;     // true / false

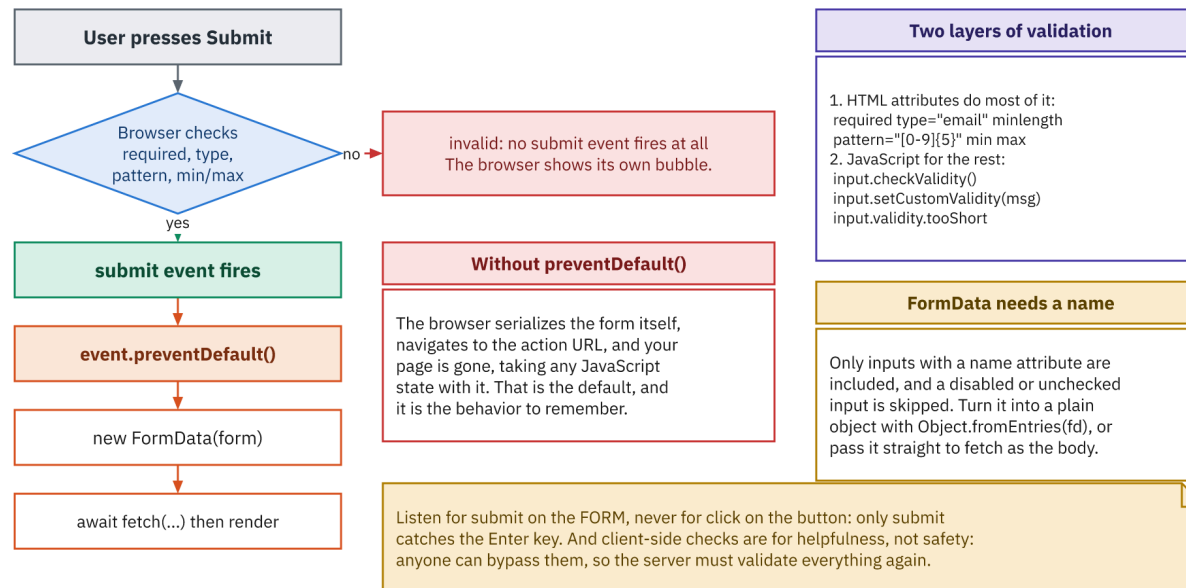
// Radio group: the group's .value is the selected one
form.elements["plan"].value;              // "free" or "pro"

// Select: .value is the chosen option
form.elements["country"].value;           // "us"
```

The `submit` Event

The Form Submit Lifecycle

The browser will navigate away unless you stop it. Everything else hangs off that one decision.



- Listen on the **form**: only `submit` catches the Enter key
- Without `preventDefault()` the browser navigates away and your page is gone

Validation Is UX, Not Security

- Code that checks input **before** you trust it
- Catches blanks, an email with no @, letters in a phone number
- A determined user can bypass anything in the browser
- **Always re-validate on the server**; client-side is for the friendly path

Validating Without Regex First

- Many rules are just plain JavaScript

```
const username = form.elements["username"].value.trim();
if (username === "") console.log("Required.");
if (username.length < 3) console.log("Too short.");

const age = Number(form.elements["age"].value);
if (Number.isNaN(age) || age < 18) console.log("Must be 18+.");

const pw = form.elements["password"].value;
const confirm = form.elements["confirm"].value;
if (pw !== confirm) console.log("Passwords do not match.");
```

- Use regex only when the rule is about the **shape** of a string

Creating a Regex and `test()`

```
const zip = /^\\d{5}$/;           // literal - preferred
const dynamic = new RegExp(word, "i"); // when it comes from a variable

zip.test("90210");    // true
zip.test("9021");     // false (only 4 digits)
zip.test("90210-1"); // false (extra chars)

"Order #4821 shipped".match(/\\d+\\/)[0]; // "4821"
```

- `regex.test(string)` → boolean, which is exactly what validation wants

Reading the Pattern Syntax

Anatomy of a Regular Expression

One pattern, every part labeled. Read it left to right, the way the engine does.

`/^[A-Z]{2}-\d{3,4}$/i`

^ Anchor: must match from the very START of the string.	[A-Z] Character class: any ONE character from the set.	{2} Quantifier: exactly two of the thing before it.	- A literal hyphen. Most characters just mean themselves.
\d Shorthand for [0-9]. Also \w word and \s space; \D \W \S negate.	{3,4} Between three and four. Also * zero or more, + one or more, ? optional.	\$ Anchor: must reach the END. ^ and \$ together mean the WHOLE string must match.	i A flag, after the closing slash. i ignore case, g find all, m multiline.
Matches: AB-123 · ab-4567 · ZZ-999		Does not: A-123 · AB-12 · AB-12345 · xAB-123	

Use a regex to check the SHAPE of a short string. Do not use one to parse HTML, and do not invent your own for email; type="email" already does it. A pattern with a g flag keeps lastIndex between calls, so reusing one across a loop can skip matches; make a fresh regex, or use String.matchAll.

- Also: `( ... )` groups, `|` means "or", and `\.` is a literal dot
- `^...$` forces the **whole** string to match, usually what you want

Common Validation Patterns

```
// Email - practical, good-enough
const email = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

// US phone - optional pieces
const phone = /^(?\\d{3}\\)?[\\-\\.\\s]?\\d{3}[\\-\\.\\s]?\\d{4}$/;

// US ZIP - 5 digits, optional +4
const zip = /^\\d{5}(-\\d{4})?$/;

// Password: lookaheads assert "somewhere ahead there is..."
const strong = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\\d){8,}$/;
strong.test("Password1"); // true
strong.test("password");  // false
```

Showing Inline Errors

- Fill a per-field container; toggle an `invalid` class

```
function showError(field, message) {  
    document.getElementById(field + "-error").textContent = message;  
    form.elements[field].classList.add("invalid");  
}  
  
function clearError(field) {  
    document.getElementById(field + "-error").textContent = "";  
    form.elements[field].classList.remove("invalid");  
}
```

Validating on Submit and on Input

- Check **every** field so the user sees all errors at once

```
form.addEventListener("submit", (event) => {
  event.preventDefault();
  let valid = true;

  const email = form.elements["email"].value.trim();
  if (!emailPattern.test(email)) {
    showError("email", "Enter a valid email.");
    valid = false;
  } else clearError("email");

  if (valid) form.reset();
});

// Live feedback as they type:
form.elements["email"].addEventListener("input", (e) => { /* ... */ });
```

novalidate and the Constraint Validation API

- Without `novalidate`, the browser blocks submit and your handler **never runs**

```
<input type="email" name="email" required minlength="8" pattern="[a-z]+">
```

```
email.checkValidity();           // true / false
email.validity.valueMissing;     // required unmet
email.validity.typeMismatch;     // type="email" not satisfied
email.validity.patternMismatch;  // pattern failed
email.validationMessage;         // the browser's own text

confirm.setCustomValidity("Passwords do not match."); // "" clears it
```

- Let HTML hold the ordinary rules; use JS for the rest

FormData: Every Field in One Call

- `new FormData(form)` scoops up **every named field**, files included
- Fields with no `name` are ignored; unchecked checkboxes are skipped

```
const data = new FormData(form);

data.get("username"); // "ada"
data.get("missing");  // null

// A repeated name - a checkbox group or <select multiple>:
data.getAll("topics"); // ["js", "css"]
data.get("topics");     // "js"  (first only)

for (const [key, value] of data.entries()) console.log(key, value);
```

FormData: Convert and Send

```
data.append("source", "web"); // add; set() replaces instead

const obj = Object.fromEntries(data.entries());
obj.topics = data.getAll("topics"); // fromEntries keeps only the LAST
```

- Send it two ways:

```
// multipart - pass FormData straight through (this carries files)
await fetch("/signup", { method: "POST", body: data });

// JSON - stringify the object (no files)
await fetch("/signup", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(obj),
});
```

JSON

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

What Is JSON?

- **JSON** = JavaScript Object Notation
- Lightweight, text-based format for storing and exchanging data
- Inspired by JS object literals; now a language-independent standard
- The format most web APIs speak; used in config files and `localStorage`
- Core idea: **serialize** (value to text) and **parse** (text back to value)

What JSON Looks Like

```
{  
  "name": "Alice",  
  "age": 30,  
  "isAdmin": false,  
  "roles": ["editor", "reviewer"],  
  "address": { "city": "Seattle", "zip": "98101" },  
  "manager": null  
}
```

- Looks like a JS object, but it is **text**, not live code, with stricter rules

JSON Value Types

- **string**: always double quotes: `"hello"`
- **number**: `42`, `3.14`, `-7` (no `NaN`, no `Infinity`)
- **boolean**: `true` or `false`
- **null**: `null`
- **object**: `{ "key": value, ... }`
- **array**: `[ value, value, ... ]`
- Objects and arrays nest to any depth

Rules That Trip People Up

```
{
  name: "Alice",           // keys must be in double quotes
  'city': 'Seattle',       // single quotes not allowed
  "age": 30,               // no trailing comma
  "greet": function () {}, // no functions
  "missing": undefined,    // no undefined
  // comments not allowed either
}
```

- Double quotes on every key/string, no trailing commas, no comments
- No functions, `undefined`, `NaN`, or `Infinity`; dates are written as strings

JSON vs XML

```
{ "user": { "name": "Alice", "roles": ["editor", "reviewer"] } }
```

```
<user>  
  <name>Alice</name>  
  <roles><role>editor</role><role>reviewer</role></roles>  
</user>
```

- JSON is **less verbose**: no repeated closing tags
- JSON has **native arrays**; parses straight into JS objects
- XML still wins for attributes, namespaces, mixed markup

JSON.stringify: Object to Text

```
const user = { name: "Alice", age: 30, isAdmin: false };

const text = JSON.stringify(user);
console.log(text);
// {"name":"Alice","age":30,"isAdmin":false}

console.log(typeof text); // "string"
```

- `JSON.stringify(value, replacer, space)`
- Compact form for storing/sending data

Pretty-Printing with **space**

```
const user = { name: "Alice", roles: ["editor", "reviewer"] };

console.log(JSON.stringify(user, null, 2));
// {
//   "name": "Alice",
//   "roles": [
//     "editor",
//     "reviewer"
//   ]
// }
```

- Third argument: number of spaces (or a string) per indent level
- Use the pretty form for logs and files people read

Filtering with **replacer**

```
const user = { name: "Alice", age: 30, password: "s3cret" };

// Array of keys to keep:
JSON.stringify(user, ["name", "age"]);
// {"name":"Alice","age":30}

// Function: return undefined to drop a key:
JSON.stringify(user, (key, value) =>
  key === "password" ? undefined : value);
// {"name":"Alice","age":30}
```

- Handy for stripping sensitive fields before sending data

Customizing with `toJSON`

```
const event = { title: "Launch", when: new Date("2026-07-07T15:00:00Z") };
JSON.stringify(event);
// {"title":"Launch","when":"2026-07-07T15:00:00.000Z"}

class Money {
  constructor(cents) { this.cents = cents; }
  toJSON() { return (this.cents / 100).toFixed(2); }
}
JSON.stringify({ price: new Money(1999) }); // {"price":"19.99"}
```

- If a value has `toJSON()`, stringify serializes its result; this is how `Date` works

JSON.parse: Text to Object

```
const text = '{"name":"Alice","roles":["editor","reviewer"]}';

const user = JSON.parse(text);
console.log(user.name);      // "Alice"
console.log(user.roles[0]);  // "editor"
```

- Invalid JSON **throws**; wrap untrusted input in `try/catch`

```
try {
  JSON.parse("{ not valid }");
} catch (error) {
  console.log("Could not parse:", error.message);
}
```

Transforming with `reviver`

```
const text = '{"title":"Launch","when":"2026-07-07T15:00:00.000Z"}';
const isoDate = /^\\d{4}-\\d{2}-\\d{2}T/;

const event = JSON.parse(text, (key, value) =>
  typeof value === "string" && isoDate.test(value)
    ? new Date(value)
    : value);

console.log(event.when instanceof Date); // true
```

- `reviver` runs for every key/value pair; its return becomes the value
- Classic use: revive ISO strings back into real `Date` objects

Common Pitfalls

```
// Dates come back as strings:
const round = JSON.parse(JSON.stringify({ when: new Date() }));
round.when instanceof Date; // false - it is a string

// Circular references throw:
const node = { name: "A" };
node.self = node;
JSON.stringify(node); // TypeError: Converting circular structure

// undefined / functions silently disappear:
JSON.stringify({ a: undefined, b: () => {}, c: 1 }); // {"c":1}
JSON.stringify([undefined, function () {}, 1]);      // [null,null,1]
```

Where JSON Is Used

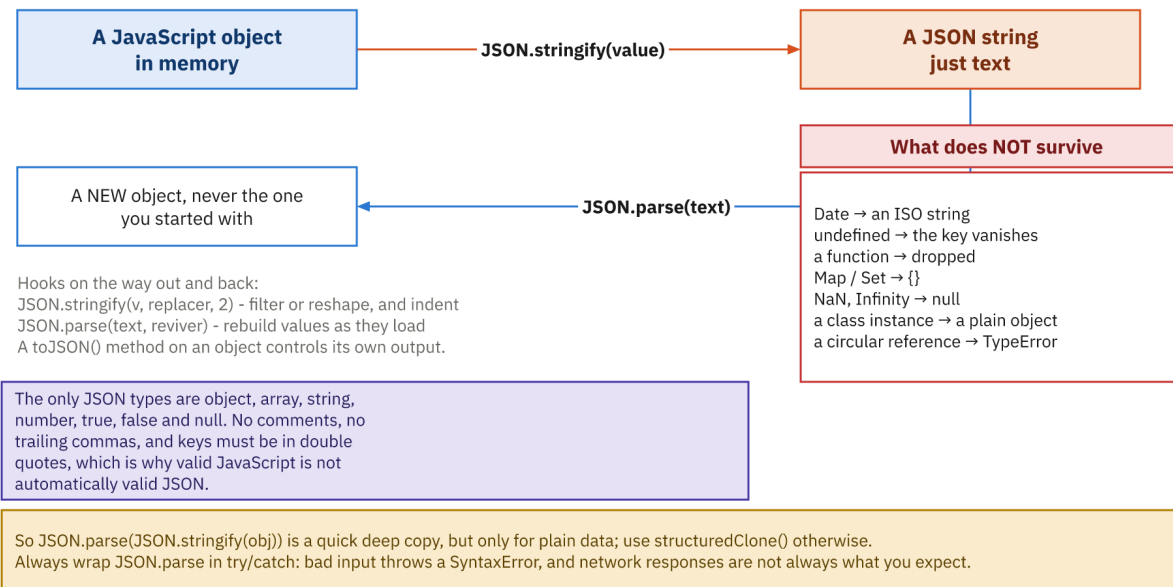
- **Web APIs:** most REST APIs speak JSON; `fetch` gives `response.json()`
- **Config files:** `package.json`, `tsconfig.json`, and many tool configs
- **Browser storage:** only holds strings, so stringify in, parse out

```
localStorage.setItem("settings", JSON.stringify({ theme: "dark" }));  
const saved = JSON.parse(localStorage.getItem("settings"));  
console.log(saved.theme); // "dark"
```


Out and Back Again

The JSON Round Trip

JSON is TEXT. Going out and coming back is not lossless: know what does not survive.



- Object to text and back, but the trip is not lossless
- Dates become strings, **undefined** vanishes, circular references throw

TypeScript and Type Annotations

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

TypeScript Is JavaScript with Types

- A language built on top of JavaScript that adds a **type system**
- Types describe the shape of your data and are checked **before** code runs
- Every valid JavaScript program is already a valid TypeScript program
- You still write JavaScript; you just also write down your intentions
- Files use `.ts` (or `.tsx` for React)

Why Bother? Errors at Your Desk

```
function total(price, quantity) { return price * quantity; }  
total("10", 5); // 50 - worked by accident; "10" was coerced
```

```
function total(price: number, quantity: number): number {  
    return price * quantity;  
}  
total("10", 5);  
// Error: Argument of type 'string' is not assignable to  
//      parameter of type 'number'.
```

- The bug surfaces **in the editor**, not in production
- The editor also gains real autocomplete, safe renames, go-to-definition

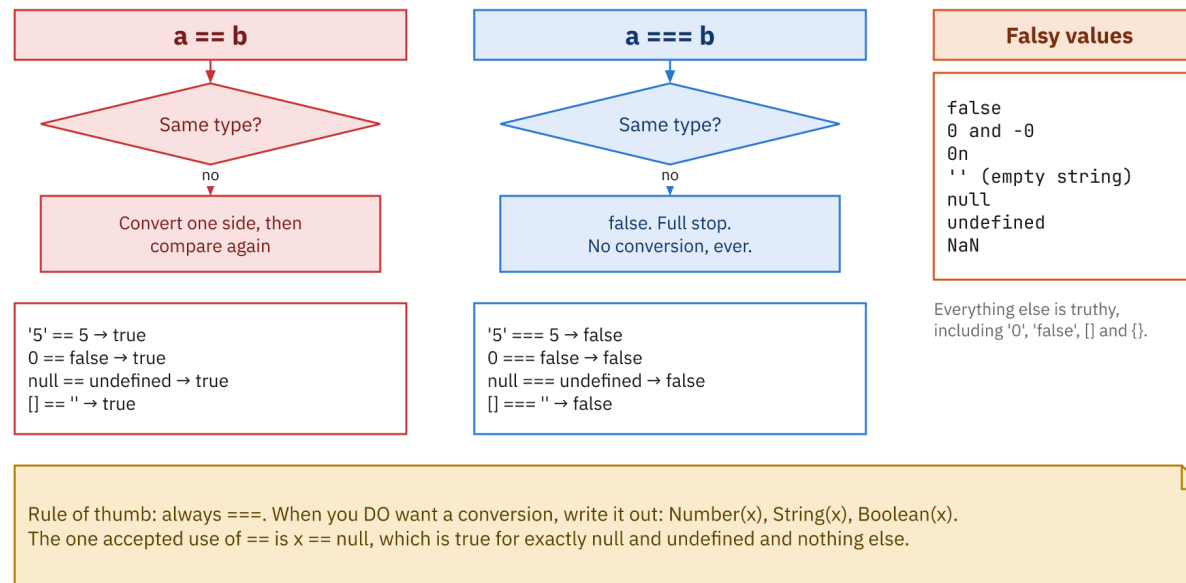
Static vs Dynamic, Strong vs Loose

- **Dynamic** (JavaScript): types checked at **runtime**; a variable can change type
- **Static** (TypeScript): types checked **ahead of time**, before the program runs
- **Loosely typed** (JavaScript): freely **coerces** between types
- **Strongly typed**: resists silent conversions and expects you to be explicit
- These are two independent axes; JavaScript is dynamic *and* loose
- TypeScript makes it static; it stays loose at runtime

Type Coercion Is Still There

Coercion and Equality

`==` converts before it compares. `===` never converts. Use `===` and convert on purpose.

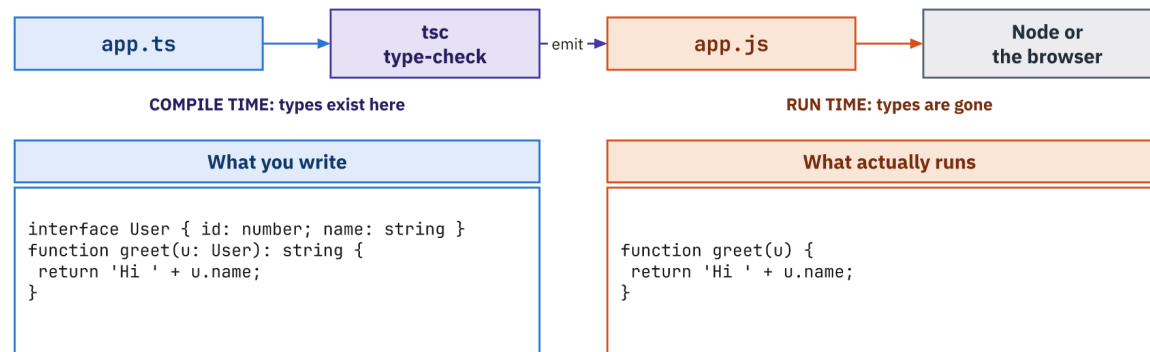


- TypeScript **flags** the risky ones (`"5" - 3`, `0 == ""`) at compile time
- Nothing changes at run time; the coercion rules are still JavaScript's

Types Are Erased Before It Runs

The TypeScript Pipeline

Types are checked, then ERASED. Nothing you write in a type annotation survives to run time.



The interface vanished entirely: an interface is a compile-time shape, not an object. So you cannot test a type with `typeof` or `instanceof`, and data crossing the boundary (JSON from a server, a form field, `localStorage`) is NOT checked. Validate it yourself.

Why bother, then? Because the check happens BEFORE you ship: a typo, a wrong argument order, or a forgotten null is a red squiggle in the editor instead of a bug in production. TypeScript is a superset: every valid `.js` file is already valid `.ts`. In this course we run `.ts` straight through `tsx`, which strips the types without type-checking; `tsc --noEmit` does the checking.

- Node and the browser only ever run plain JavaScript, no runtime cost
- So a type cannot check data that only arrives at run time. Validate it yourself

Running TypeScript

```
npx tsx app.ts      # strip types and run - what this course uses
node app.ts         # Node 24+ can too, but see below
npx tsc --noEmit     # full type-check, emit nothing
npx tsc app.ts       # type-check AND emit app.js
```

- Neither `tsx` nor `node` type-checks; both just strip and run
- Node **strips only**: `enum`, `namespace`, and `constructor(private x: number)` all fail with `ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX`. Two demos this week use the last one, so we use `tsx`
- `tsconfig.json` holds the options; `"strict": true` is the one that matters

Type Annotations on Variables

```
let age: number = 30;  
let firstName: string = "Ada";  
let isAdmin: boolean = false;  
  
age = "thirty";  
// Error: Type 'string' is not assignable to type 'number'.
```

- A **type annotation** is a colon and a type after the name
- Once annotated, the variable is locked to that type

Primitive Types

```
let count: number = 42;      // ints and floats are both `number`  
let title: string = "Hello";  
let done: boolean = true;  
let nothing: null = null;  
let missing: undefined = undefined;  
let big: bigint = 100n;  
let key: symbol = Symbol("id");
```

- Type names are **lowercase**
- No separate integer/float type; every number is `number`

Type Inference

```
let city = "Boston"; // inferred as string
city = 42;           // Error: 'number' not assignable to 'string'

let total = 0;        // inferred as number
const PI = 3.14159;  // inferred as the literal 3.14159
```

- TypeScript **infers** the type from the assigned value
- Let TS infer obvious locals; annotate parameters and boundaries
- `let city: string = "Boston"` is redundant; inference already knows

any, unknown, never

```
let data: any = 5;
data.foo.bar.baz;    // no error - but crashes at runtime

let input: unknown = fetchSomething();
input.toUpperCase();    // Error: 'input' is of type 'unknown'
if (typeof input === "string") input.toUpperCase(); // OK after check

function fail(message: string): never {
  throw new Error(message);    // never returns normally
}
```

- `any` opts **out** of checking (avoid); `unknown` is the safe version

Arrays and Tuples

```
let scores: number[] = [90, 85, 100];
let names: Array<string> = ["Ada", "Alan"]; // equivalent generic form

scores.push("hello");
// Error: 'string' is not assignable to 'number'.

// A tuple: fixed length, a type per position
let entry: [string, number] = ["age", 30];
entry[0].toUpperCase(); // position 0 is a string
entry[1].toFixed(2);      // position 1 is a number
```

Union and Literal Types

```
let id: number | string;
id = 101;      // OK
id = "A-101";  // OK
id = true;     // Error: 'boolean' not assignable

let direction: "up" | "down" | "left" | "right";
direction = "up";      // OK
direction = "north";   // Error: not one of the allowed literals
```

- A **union** (`|`) means "one of these types"
- A **literal type** narrows to exact values, TS's lightweight enum

Typing Function Parameters and Returns

```
function add(a: number, b: number): number {  
    return a + b;  
}  
add(2, "3"); // Error: 'string' not assignable to 'number'  
  
function greet(name: string): string {  
    return `Hi, ${name}`;  
}  
  
const multiply = (a: number, b: number): number => a * b;
```

- Annotate each **parameter**; the return type is often inferred but documents intent

Optional, Default, and `void`

```
function greet(name: string, title?: string): string {  
    return title ? `Hello, ${title} ${name}` : `Hello, ${name}`;  
}  
  
function increment(value: number, step = 1): number {  
    return value + step; // default makes step optional  
}  
  
function logMessage(message: string): void {  
    console.log(message); // returns nothing  
}
```

- `?` = optional (must follow required params); a default supplies a fallback
- `void` = returns nothing; differs from `never` (which never finishes)

Interfaces and Type Aliases

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Object Types and Type Aliases

```
// Inline shape - works, but tedious to repeat:
let user: { name: string; age: number } = { name: "Ada", age: 30 };

// A type alias names ANY type:
type User = { name: string; age: number };
const ada: User = { name: "Ada", age: 30 };

type ID = number | string;           // union
type Point = [number, number];       // tuple
type Direction = "up" | "down";     // literal union
```

- Only aliases can name unions, tuples, and primitives

Interfaces

```
interface User {  
  name: string;  
  age: number;  
}  
  
const ada: User = { name: "Ada", age: 30 };
```

- An **interface** describes only **object-like** shapes (objects, classes, functions)
- Cannot name a union or primitive
- In exchange: `extends` and declaration merging

Optional and Readonly Properties

```
interface User {  
  name: string;  
  nickname?: string; // optional - may be absent  
}  
  
interface Account {  
  readonly id: number;  
  balance: number;  
}  
  
const acct: Account = { id: 1, balance: 100 };  
acct.balance = 150; // OK  
acct.id = 2;        // Error: 'id' is read-only
```

- Both `type` and `interface` support `?` and `readonly`
- `readonly` is compile-time only; it does not freeze the object at runtime

Extending and Merging

```
interface Animal { name: string; }
interface Dog extends Animal { breed: string; } // inherits name

type Animal2 = { name: string };
type Dog2 = Animal2 & { breed: string };          // intersection

interface Box { width: number; }
interface Box { height: number; }                // merges - needs both
```

- Interfaces extend with **extends**; type aliases combine with **&**
- Only interfaces support **declaration merging** (same name twice)

Index Signatures

```
interface ScoreBoard {  
  [player: string]: number;  
}  
  
const scores: ScoreBoard = {};  
scores["Ada"] = 90;  
scores["Alan"] = 85;  
scores["Grace"] = "high";  
// Error: Type 'string' is not assignable to type 'number'.
```

- Describes objects whose **keys aren't known ahead of time**
- Reads as "any string key maps to a number value", types dictionaries

Structural Typing

- TypeScript matches on **shape**, not on name

```
interface Point { x: number; y: number; }

function print(p: Point) { console.log(p.x, p.y); }

const thing = { x: 1, y: 2, label: "extra" };
print(thing); // OK - it HAS x and y

print({ x: 1, y: 2, label: "extra" });
// Error: object literals may only specify known properties
```

- A variable with extra properties passes; an inline **literal** does not
- That extra check exists to catch typos in option objects

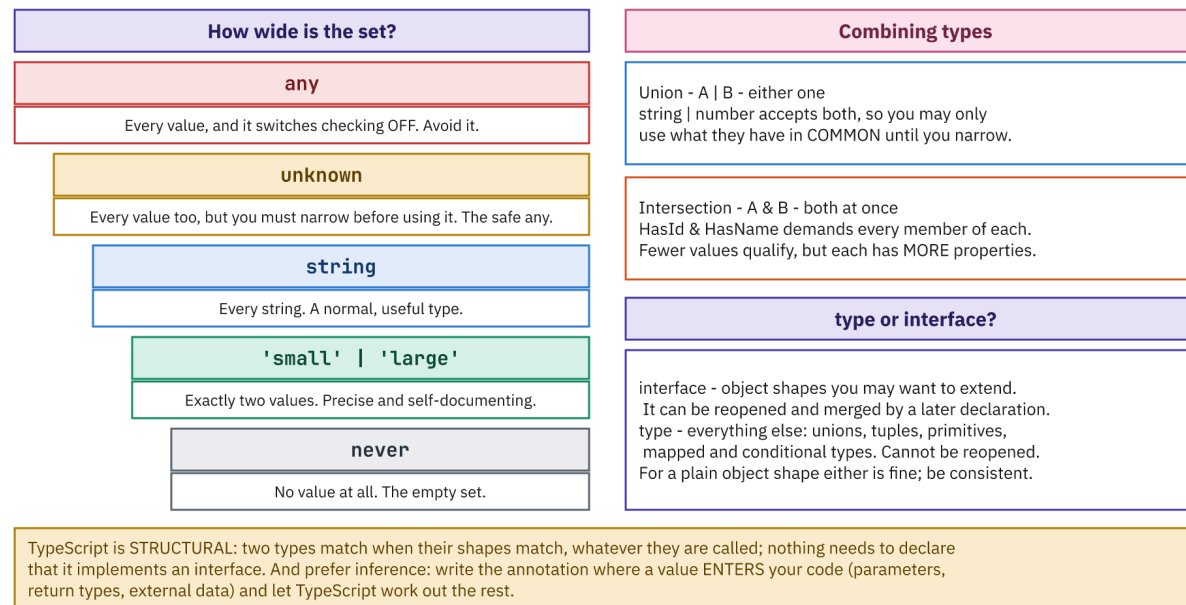
`type` vs `interface`: When to Use Which

- Reach for `interface` for the shape of an **object or class**, especially if it will be **extended** or **implemented**
- Reach for `type` for what an interface can't express: unions, tuples, primitives, composed types
- Reasonable default: **interfaces for object/class shapes, aliases for the rest**
- Above all, **be consistent** within a codebase

How Wide Is the Set?

The TypeScript Type Space

Think of a type as the SET of values it allows. Unions widen that set; intersections narrow it.



- A type is the **set** of values it allows
- Unions widen that set; intersections narrow it

Classes and Abstract Classes

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Classes in TypeScript

```
class Point {  
  x: number;  
  y: number;  
  constructor(x: number, y: number) {  
    this.x = x;  
    this.y = y;  
  }  
  distanceFromOrigin(): number {  
    return Math.sqrt(this.x ** 2 + this.y ** 2);  
  }  
}  
new Point(3, 4).distanceFromOrigin(); // 5
```

- TS requires you to **declare fields** and checks the constructor initializes them

Parameter Properties

```
class Point {  
  constructor(public x: number, public y: number) {}  
  // Declares x and y AND assigns this.x = x, this.y = y  
}  
  
const p = new Point(3, 4);  
p.x; // 3
```

- Put an access modifier on a constructor parameter, TS creates and assigns the field
- A shorthand for the very common declare-then-assign pattern

Access Modifiers

```
class BankAccount {  
  private balance = 0;  
  deposit(amount: number): void { this.balance += amount; }  
  getBalance(): number { return this.balance; }  
}  
  
const acct = new BankAccount();  
acct.deposit(100);  
acct.balance; // Error: 'balance' is private
```

- **public** (default) everywhere · **private** this class only
- **protected** this class + subclasses · **readonly** set once in constructor

private vs #private

```
class A {  
    private secret = 1;    // TypeScript only - erased at compile time  
    #realSecret = 2;       // JavaScript - enforced at RUNTIME  
}  
  
const a = new A() as any;  
a.secret;                // 1 - the check was compile-time only!  
a["#realSecret"];        // undefined - genuinely unreachable
```

- `private` stops *your code* compiling; `#` stops *any code* at runtime
- Use `#` when the guarantee has to survive the type erasure

Implementing an Interface

```
interface Shape {  
    area(): number;  
    name: string;  
}  
  
class Circle implements Shape {  
    name = "circle";  
    constructor(private radius: number) {}  
    area(): number { return Math.PI * this.radius ** 2; }  
}
```

- **implements** makes a class satisfy an interface's contract
- TS reports any missing member; a class can implement **multiple** interfaces
- **implements** only checks the shape; it inherits no code

Abstract Classes

```
abstract class Shape {  
  abstract area(): number;      // no body - subclasses MUST implement  
  describe(): string {          // concrete, shared method  
    return `This shape has area ${this.area()}`;  
  }  
}  
  
new Shape();  
// Error: Cannot create an instance of an abstract class.
```

- An **abstract class** cannot be instantiated; it exists to be extended

Abstract Methods

```
class Rectangle extends Shape {  
  constructor(private width: number, private height: number) {  
    super();  
  }  
  area(): number { return this.width * this.height; } // required  
}  
  
const r = new Rectangle(3, 4);  
r.area();      // 12  
r.describe();  // "This shape has area 12" (inherited)
```

- Every concrete subclass **must** implement each abstract method

Abstract Class vs Interface

- **Interface**: purely a shape, no implementation; a class can implement **many**
- **Abstract class**: can hold **shared code** *and* enforce abstract members, but a class extends only **one**
- Need to **share code** among related classes? Use an **abstract class**
- Only need to **guarantee a shape**? Use an **interface**
- An interface disappears at compile time; an abstract class is real JS output

What TypeScript Is Typing

Class Anatomy

Where each part actually lives: fields on the instance, methods on the prototype, statics on the class itself.

```
class Animal {
  static kingdom = 'Animalia';
  #secret = 'hidden';
  constructor(name) { this.name = name; }
  speak() { return this.name + ' makes a sound'; }
  get label() { return this.name.toUpperCase(); }
}
class Dog extends Animal {
  constructor(name, breed) {
    super(name);
    this.breed = breed;
  }
  speak() { return super.speak() + ' (a bark)'; }
}
```

class is syntax over the prototype chain, not a new object model: the same lookup rules apply. Two things it does add: #private fields are enforced by the language (unlike a leading underscore), and the whole class body runs in strict mode.

On the INSTANCE

name, breed
#secret
One copy per object. Anything assigned to this in the constructor.

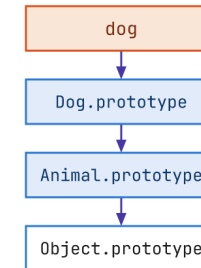
On the PROTOTYPE

speak(), get label
ONE copy, shared by every instance. This is why methods cost nothing per object.

On the CLASS

Animal.kingdom
Statics belong to the class, not to any instance. dog.kingdom is undefined.

What extends actually does



extends just links these. super(name) runs the parent constructor, and you must call it before touching this.

- The same instance / prototype / class layout, now checked
- `abstract` exists only at compile time; it is erased like every other type

Generics and Utility Types

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Why Generics?

```
// Locked to one type - reusable for nothing else:  
function identityNumber(value: number): number { return value; }  
  
// any - reusable, but no safety:  
function identity(value: any): any { return value; }  
identity("hello").toFixed(2); // no compile error, CRASHES at runtime
```

- Generics give the best of both: **reuse AND type safety**

Generic Functions

```
function identity<T>(value: T): T {  
    return value;  
}  
  
const a = identity("hello"); // T is string  
const b = identity(42);      // T is number  
b.toUpperCase();  
// Error: 'toUpperCase' does not exist on type 'number'.  
  
const c = identity<string>("world"); // T set explicitly
```

- Declare a **type parameter** in `<T>`; TS **infers** it from the argument
- Supply it explicitly when there is nothing to infer from (an empty array)

Constraints with `extends`

```
function longest<T extends { length: number }>(a: T, b: T): T {  
    return a.length >= b.length ? a : b;  
}  
  
longest("apple", "kiwi");    // OK - strings have .length  
longest([1, 2, 3], [4, 5]); // OK - arrays have .length  
longest(10, 20);  
// Error: 'number' not assignable to '{ length: number }'.
```

- A **constraint** limits `T`, "any type, as long as it has `.length`"
- Without it, you could not safely read `.length` inside the function

Generic Interfaces and Aliases

```
interface ApiResponse<T> {  
  status: number;  
  data: T;  
}  
  
const res: ApiResponse<{ name: string }> = {  
  status: 200,  
  data: { name: "Ada" },  
};  
res.data.name; // string - the T is remembered  
  
type Pair<K, V> = { key: K; value: V };  
const entry: Pair<string, number> = { key: "age", value: 30 };
```


Generic Classes

```
class Box<T> {  
  private contents: T;  
  constructor(value: T) { this.contents = value; }  
  get(): T { return this.contents; }  
  set(value: T): void { this.contents = value; }  
}  
  
const numberBox = new Box(123); // Box<number>  
numberBox.set("no");  
// Error: 'string' not assignable to 'number'.
```

- Exactly how built-in `Array<T>` and `Map<K, V>` are typed

A Generic `Stack<T>`

```
class Stack<T> {  
  private items: T[] = [];  
  push(item: T): void { this.items.push(item); }  
  pop(): T | undefined { return this.items.pop(); }  
  peek(): T | undefined { return this.items[this.items.length - 1]; }  
  get size(): number { return this.items.length; }  
}  
  
const words = new Stack<string>();  
words.push("hello");  
words.peek()?.toUpperCase(); // "HELLO"
```

- `pop` returns `T | undefined`; the empty case is in the type

What Are Utility Types?

- Built-in **generic helpers** that transform an existing type into a new one
- Instead of hand-writing a variant, you **derive** it
- Keeps types **DRY**: change the original, every derived type updates

```
interface User {  
  id: number;  
  name: string;  
  email: string;  
  age: number;  
}
```

Partial and Required

```
type PartialUser = Partial<User>;  
// { id?: number; name?: string; email?: string; age?: number }  
  
function updateUser(id: number, changes: Partial<User>): void {}  
updateUser(1, { email: "new@example.com" }); // OK - any subset  
  
interface Config { host?: string; port?: number; }  
type FullConfig = Required<Config>;  
// { host: string; port: number } - both now required
```

Readonly, Pick, and Omit

```
type FrozenUser = Readonly<User>;
const u: FrozenUser = { id: 1, name: "Ada", email: "a@example.com", age: 30 };
u.name = "Grace"; // Error: read-only

type UserPreview = Pick<User, "id" | "name">;
// { id: number; name: string }

type NewUser = Omit<User, "id">;
// { name: string; email: string; age: number }
```

- **Pick** keeps only named keys · **Omit** removes named keys

Record: Build a Map Type

```
type Roles = "admin" | "editor" | "viewer";

type Permissions = Record<Roles, boolean>;
const perms: Permissions = {
  admin: true, editor: true, viewer: false,
}; // all three keys required - miss one and it will not compile

type PageViews = Record<string, number>;
const views: PageViews = { home: 100, about: 40 };
```

ReturnType and Composing

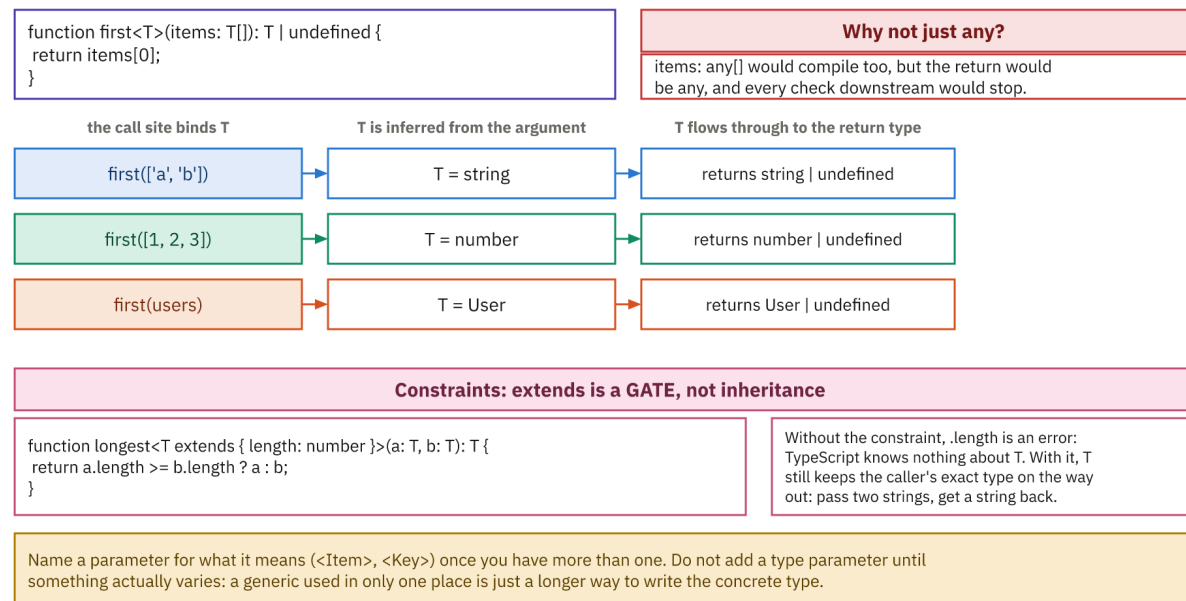
```
function createUser(name: string) {  
  return { id: 1, name, active: true };  
}  
  
type CreatedUser = ReturnType<typeof createUser>;  
// { id: number; name: string; active: boolean }  
  
// Utility types compose:  
type DraftUser = Partial<Omit<User, "id">>;  
// { name?: string; email?: string; age?: number }
```

- `ReturnType<typeof fn>` extracts a return type, and stays in sync
- Because all derive from `User`, editing `User` once updates every derived type

How **T** Flows

How a Type Parameter Flows

T is a placeholder filled in at the CALL. It carries the real type through to the return, so nothing is lost.

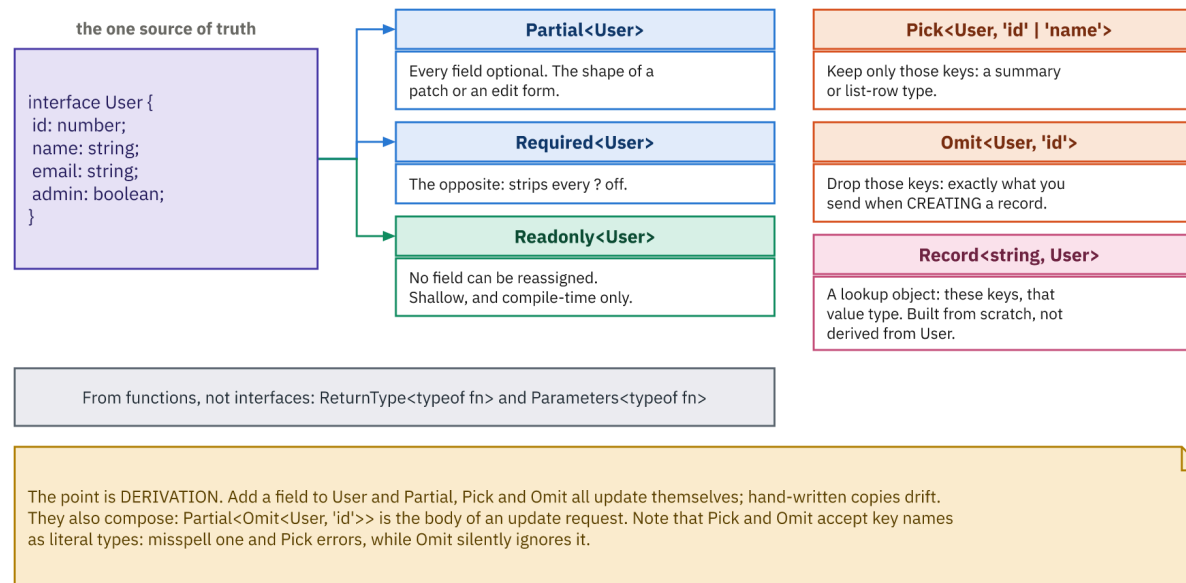


- **T** is bound at the call site and carried through to the return
- That is the whole difference from **any**, which forgets

Utility Types as Transforms

Utility Types as Transforms

One source of truth in, several derived shapes out. Change the interface and every one of them follows.



- One source of truth in, several derived shapes out
- Add a field and every derived type follows; hand-written copies drift

Maps and Sets

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Reserve pass. Runs only if the class finishes 01-30. Note that this is the only coverage of outline IV.G, and Module 09's exit ticket asks about Map.

Map: Keyed Collections Done Right

- Key/value pairs; keys can be **any** value; order preserved

```
let prices = new Map();
prices.set("espresso", 3.5); // set(key, value)
prices.set("latte", 4.5);

prices.get("latte"); // 4.5
prices.get("mocha"); // undefined
prices.has("mocha"); // false
prices.size;          // 2    (a property, not size())
prices.delete("espresso");
```

- `set` returns the map, so calls chain

Why **Map** Over an Object?

```
let obj = {};  
obj[1] = "a";  
obj["1"] = "b";  
obj[1];           // "b" - number key coerced to string!  
  
let map = new Map();  
map.set(1, "a").set("1", "b");  
map.get(1);       // "a" - kept distinct
```

- Instant **.size**, no inherited-key surprises, guaranteed insertion order
- Use a plain object for fixed records; a **Map** for dynamic/non-string keys

Iterating a Map

- `for...of` yields `[key, value]` pairs, destructure them

```
let prices = new Map([
  ["espresso", 3.5],
  ["latte", 4.5],
]);

for (let [item, price] of prices) {
  console.log(`${item}: ${price}`);
}

[...prices.keys()]; // ["espresso", "latte"]
[...prices.values()]; // [3.5, 4.5]
```

- Seed from an array of `[key, value]` pairs

Counting With a **Map**

- The canonical use: a tally where the keys are not known up front

```
function tally(items) {  
  const counts = new Map();  
  for (const item of items) {  
    counts.set(item, (counts.get(item) ?? 0) + 1);  
  }  
  return counts;  
}  
  
tally(["js", "css", "js"]); // Map { "js" => 2, "css" => 1 }  
  
Object.fromEntries(tally(["js", "js"])); // { js: 2 }
```

Set: Unique Values

```
let tags = new Set();
tags.add("coffee");
tags.add("hot");
tags.add("coffee"); // ignored - already present

tags.has("hot"); // true
tags.size;       // 2
tags.delete("hot");
```

- Adding a value already present does nothing, no error, no duplicate

Deduplicating and Iterating a Set

- The single most common use: remove repeats in one line

```
let raw = [1, 2, 2, 3, 3, 3, 4];  
[...new Set(raw)]; // [1, 2, 3, 4]  
  
new Set(raw).size; // 4 - a distinct count, no loop  
  
let colors = new Set(["red", "green", "blue"]);  
for (let color of colors) {  
  console.log(color); // red, green, blue  
}
```

- `set.has(x)` stays fast as the collection grows; `array.includes(x)` does not

Set Math

- Modern methods for comparing two sets

```
const mine    = new Set(["js", "css", "react"]);
const theirs = new Set(["css", "vue", "js"]);

[...mine.intersection(theirs)]; // ["js", "css"] - in both
[...mine.union(theirs)];        // everything, no repeats
[...mine.difference(theirs)];   // ["react"]      - only mine

mine.isSubsetOf(theirs);        // false
```

- Each returns a **new** Set; the originals are untouched

Equality and Weak Collections

- Keys/values compared with `===`, **objects match by reference**

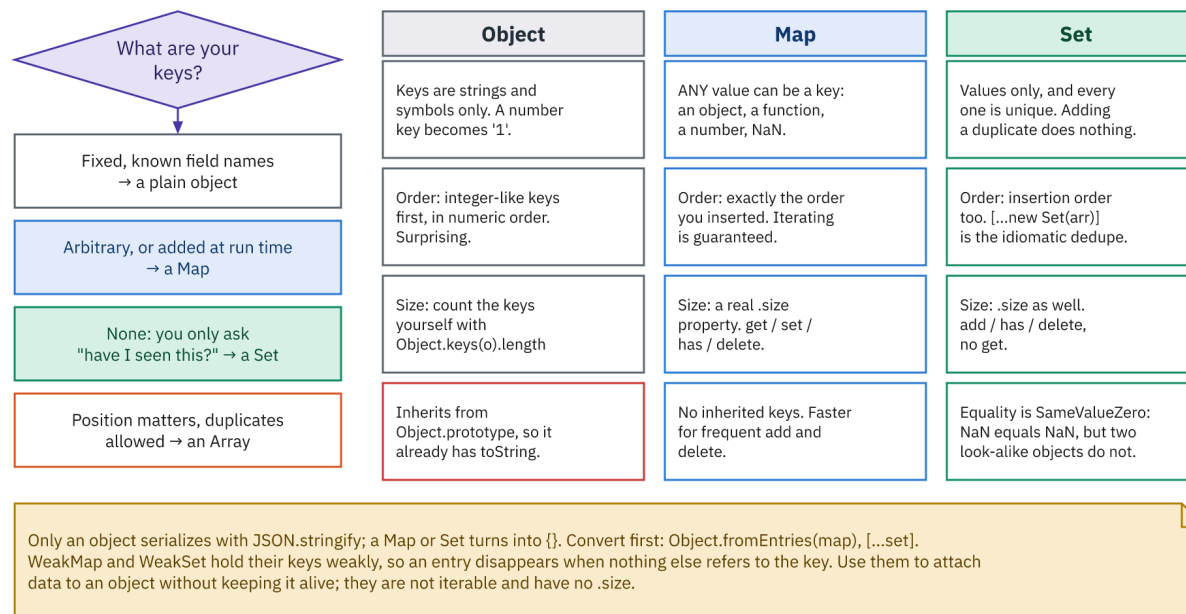
```
let s = new Set();  
s.add({ id: 1 });  
s.add({ id: 1 }); // a DIFFERENT object - both kept  
s.size; // 2
```

- `WeakMap` / `WeakSet` hold **objects** weakly (GC-friendly metadata)
- They are not iterable and have no `.size`

Which One, and Why

Map and Set vs Object and Array

Pick by the KEY you have and the question you ask.



- Any key means `Map`; no key at all means `Set`
- Only a plain object survives `JSON.stringify`; convert first

Working with Dates

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Reserve price. Runs only if the class finishes 01-30. Note that this is the only coverage of outline IV.F, JavaScript dates.

A Date Is One Number

- Under the hood: **milliseconds since Jan 1, 1970 UTC** (the Unix epoch)

```
let now = new Date();           // current date and time
let epoch = new Date(0);        // Jan 1, 1970 UTC

let birthday = new Date(2026, 6, 7); // July 7 - month 6 = July!
let fromString = new Date("2026-07-07"); // ISO, unambiguous
let withTime = new Date("2026-07-07T14:30:00");
```

- **Months are zero-based**, the most common `Date` mistake

Timestamps

```
Date.now();           // 1783382400000 - no object needed  
new Date().getTime(); // same value from an instance
```

- Ideal for measuring elapsed time

```
let start = Date.now();  
// ...do work...  
let elapsedMs = Date.now() - start;
```

Reading the Parts

- `get*` methods read components in **local time**

```
let d = new Date("2026-07-07T14:30:15");  
d.getFullYear(); // 2026  
d.getMonth();    // 6   (July - zero-based!)  
d.getDate();     // 7   (day of MONTH, 1-based)  
d.getDay();      // 2   (day of WEEK, 0 = Sunday)  
d.getHours();    // 14
```

- Watch the mix-up: `getDate()` = day of month, `getDay()` = day of week

Changing Parts (Dates Mutate)

```
let d = new Date("2026-07-07");  
d.setFullYear(2027);  
d.setMonth(11); // December  
d.setDate(25);
```

- `set*` mutates in place; clone first to keep the original

```
let original = new Date("2026-07-07");  
let copy = new Date(original); // independent clone  
copy.setDate(14); // original untouched
```


Date Arithmetic

- `setDate` handles overflow for you; no month-length math

```
function addDays(date, n) {  
  const copy = new Date(date); // clone, don't mutate the input  
  copy.setDate(copy.getDate() + n);  
  return copy;  
}  
  
const jan30 = new Date(2026, 0, 30);  
addDays(jan30, 5).toDateStr(); // "Wed Feb 04 2026"
```

- Rolling past the end of a month just works

Formatting for People

```
let d = new Date("2026-07-07T14:30:00");
d.toLocaleDateString(); // "7/7/2026"
d.toLocaleTimeString(); // "2:30:00 PM"

d.toLocaleDateString("en-US", {
  weekday: "long", year: "numeric",
  month: "long", day: "numeric",
}); // "Tuesday, July 7, 2026"
```

- Build a reusable `Intl.DateTimeFormat` when formatting many dates

Date Math and ISO Strings

- Subtract dates for a gap in milliseconds, then scale

```
let start = new Date("2026-07-01");  
let end = new Date("2026-07-07");  
let diffDays = (end - start) / (1000 * 60 * 60 * 24); // 6
```

- Store/transmit as **ISO 8601**; format for display only at the end

```
new Date("2026-07-07T14:30:00Z").toISOString();  
// "2026-07-07T14:30:00.000Z" (Z = UTC)
```

- Date-only string parses as **UTC**; date-time without **Z** parses as **local**

Looking Ahead: **Temporal**

- **Date** mutates, has zero-based months, and no time-zone support
- **Temporal** fixes all three, finished at TC39, expected in **ES2027**

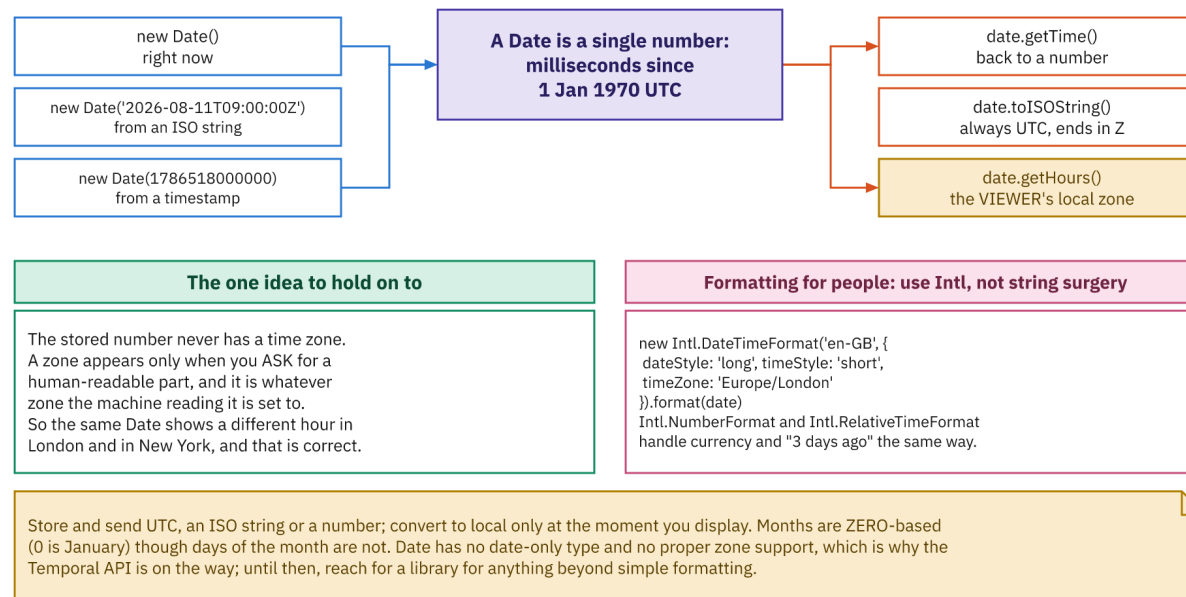
```
// Immutable, 1-based months, explicit time zones  
Temporal.PlainDate.from("2026-07-07").add({ days: 5 });
```

- **Not in Node yet**, and not in every browser, a polyfill is required today
- Know it exists and what it fixes; keep using **Date** for now

One Number, Many Faces

The Date and Time Model

A Date is one number. Everything else (the year, the string, the time zone) is presentation.



- A `Date` is milliseconds since 1970; the rest is presentation
- Store UTC, convert to local only when you display

Function Recursion

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Reserve path. Runs only if the class finishes 01-30. Recursion is in neither the outline nor any exit ticket, so this is the one with no cost to skipping.

What Is Recursion?

- A function that **calls itself** on a smaller version of the problem
- Every recursive function needs two parts:
 - A **base case** that returns without recursing
 - A **recursive case** that moves *toward* the base case

```
function countdown(n) {  
  if (n <= 0) return;    // base case - stop  
  console.log(n);  
  countdown(n - 1);      // recursive case - smaller  
}  
  
countdown(3); // 3, 2, 1
```

Forget the Base Case, Blow the Stack

```
function forever(n) {  
  console.log(n);  
  forever(n - 1);  // nothing ever stops it  
}  
  
forever(3);  
// RangeError: Maximum call stack size exceeded
```

- Each call takes a frame on the **call stack**; the stack is finite
- Two failure modes: no base case, or a step that never reaches it
- Check the base case **first**, before doing any work

Factorial: The Classic

```
function factorial(n) {  
  if (n <= 1) return 1;      // base case  
  return n * factorial(n - 1); // recursive case  
}  
  
factorial(5); // 120
```

- Read it as a definition: $5! = 5 \times 4!$, and $1! = 1$
- The multiplication happens **on the way back up**, as calls return

Tracing the Calls

```
factorial(4)
  4 * factorial(3)
    3 * factorial(2)
      2 * factorial(1)
        1          <- base case reached
      2 * 1        = 2
    3 * 2          = 6
  4 * 6            = 24
```

- Calls **stack up** until the base case, then **collapse back**
- Drawing this on the board is worth more than any explanation

A Running Total

```
function sumTo(n) {  
  if (n <= 0) return 0;    // base case  
  return n + sumTo(n - 1); // recursive case  
}
```

```
sumTo(5); // 15  
sumTo(10); // 55
```

- Same shape as `factorial`, only the operation and base value differ
- Most simple recursion is this one pattern wearing different clothes

Fibonacci, Two Ways

```
// Iterative - one pass, two variables
function fibonacci(n) {
  if (n <= 0) return 0;
  if (n === 1) return 1;
  let a = 0, b = 1;
  for (let i = 2; i <= n; i++) [a, b] = [b, a + b];
  return b;
}

// Recursive - reads like the mathematical definition
function fibRecursive(n) {
  if (n <= 0) return 0;
  if (n === 1) return 1;
  return fibRecursive(n - 1) + fibRecursive(n - 2);
}
```

The Recursive Version Is Much Slower

```
fibRecursive(10); // 55 - fine  
fibRecursive(40); // 102334155 - takes SECONDS
```

- Each call spawns **two** more, so the work roughly doubles per step
- `fib(40)` makes over 300 million calls, recomputing the same values
- Memoizing it (a closure over a cache) collapses that back to linear
- **Recursion is not automatically elegant: measure it**

When Recursion Actually Wins

- Not for counting; a loop is clearer and faster
- Use it when the **data is nested to an unknown depth**

```
function flatten(arr) {  
  const out = [];  
  for (const item of arr) {  
    if (Array.isArray(item)) out.push(...flatten(item)); // recurse  
    else out.push(item);  
  }  
  return out;  
}  
  
flatten([1, [2, [3, [4]], 5]]); // [1, 2, 3, 4, 5]
```

- The loop version needs an explicit stack; the recursion just works

Where You Will Meet It

- Walking a **DOM tree**, elements contain elements
- Reading **nested JSON** from an API
- Traversing a **file system**, folders inside folders
- Any tree, menu, comment thread, or org chart

```
function countNodes(el) {  
  let total = 1;  
  for (const child of el.children) total += countNodes(child);  
  return total;  
}
```

- The rule of thumb: **loop for a list, recurse for a tree**

Down, Then Back Up

The Recursion Call Tree

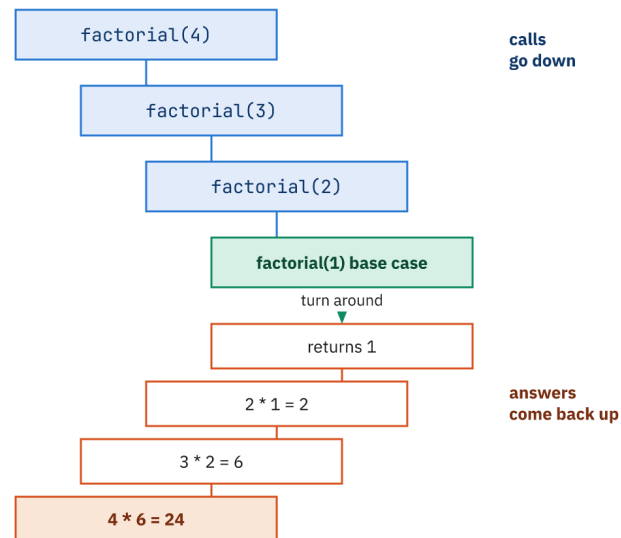
Calls go DOWN until the base case. Then answers come back UP. Nothing returns until the bottom is reached.

```
function factorial(n) {  
  if (n <= 1) return 1; // base case  
  return n * factorial(n - 1); // recursive case  
}
```

Every recursive function needs two things

1. A BASE CASE that returns without calling itself again.
 2. A step that moves CLOSER to it: here, $n - 1$.
- Miss either one and every call adds another frame until the engine gives up: `RangeError: Maximum call stack size exceeded`.

Recursion shines where the DATA is nested: a folder tree, a DOM subtree, a JSON document. For a flat list, a loop is simpler and cannot blow the stack. Each pending call holds a real stack frame, which is why depth, not total work, is the limit.



- Calls descend to the base case; nothing returns until the bottom
- Every pending call holds a stack frame, so depth is the limit

Dynamic `import()`

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Reserve path. Runs only if the class finishes `01-30`. Note that this is the only coverage of outline IX.D, and Module 06's exit ticket asks about `import()`.

The `import()` Function

- Loads modules **at runtime**, standardized in **ES2020**
- Function-like syntax vs static `import`

```
// static (ES2015)
import { something } from "./module.js";

// dynamic (ES2020)
import("./module.js").then((module) => {
  // use module
});
```

- `import()` is **asynchronous** and returns a **promise**
- It resolves to a **module namespace object** (its exports)

Using `import()` with `async/await`

```
async function loadMath() {
  try {
    const math = await import("./math.js");
    console.log(math.PI);
    console.log(math.add(2, 3));
    math.default();           // the default export
  } catch (error) {
    console.error("Failed to load module:", error.message);
  }
}
loadMath();
```

- Named exports → `math.add`; default export → `math.default`

It Can Appear Anywhere

- Unlike static `import`, `import()` works inside functions and conditions

```
async function setup() {  
  if (location.search.includes("debug")) {  
    const debug = await import("./debug-tools.js");  
    debug.enableDebugMode();  
  }  
}
```

- The module loads **only when the condition passes**
- Static import is fixed at parse time; dynamic loads on demand

The Path Can Be Built at Runtime

```
const name = userChoice;           // "chart" | "table" | "map"  
const view = await import(`./views/${name}.js`);  
view.render();
```

- Impossible with a static `import`, whose path must be a literal
- This is what powers plugin systems and route-based loading

Why Bother: Code Splitting

- Static imports all load **before** your app starts
- A large, rarely-used feature makes every visitor wait

```
// 400 KB chart library - only for the 5% who open the report
document.getElementById("report").addEventListener("click", async () => {
  const { renderChart } = await import("./charts.js");
  renderChart(data);
});
```

- Faster first load; the cost moves to the people who use the feature

A Module Evaluates Only Once

```
const a = await import("./setup.js"); // runs the module body
const b = await import("./setup.js"); // cached - body does NOT re-run

console.log(a === b); // true - same namespace object
```

- Import it ten times, it initializes once
- Safe to call in a click handler that fires repeatedly

Failure Is a Rejected Promise

```
try {  
  const mod = await import("./maybe-missing.js");  
} catch (error) {  
  console.warn("Optional feature unavailable:", error.message);  
}
```

- A static `import` of a missing file fails **before your code runs**
- A dynamic one fails **where you called it**, so you can recover
- Always wrap it: network hiccups make this a real path, not a rare one

Static and Dynamic Together

```
// main.js, loaded with <script type="module">
import { setupPage } from "./setup.js"; // needed always - static
setupPage();

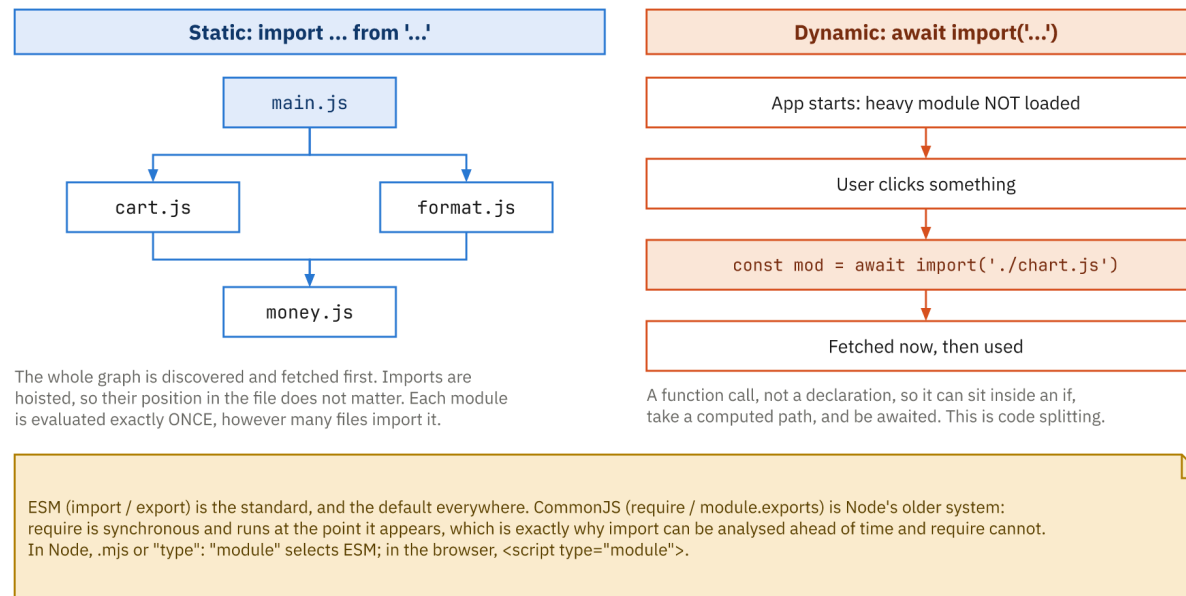
document.getElementById("debug-btn")
  .addEventListener("click", async () => {
    const debug = await import("./debug-tools.js"); // rarely - dynamic
    debug.openPanel();
  });
```

- Core code up front, optional features on demand

Static and Dynamic, Together

The Module Graph

Static imports are resolved before any of your code runs. Dynamic `import()` happens later, and returns a promise.



- `import()` is a function call, so it can sit inside an `if`
- It returns a promise; that is what makes code splitting possible

Narrowing and Discriminated Unions

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Reserve path. Runs only if the class finishes 01-30. Module 12's exit ticket asks how you safely use a `string | number`, which *is* narrowing.

A Union Is Only Useful Once Narrowed

```
function format(id: number | string) {  
  id.toUpperCase();  
  // Error: 'toUpperCase' does not exist on type 'number'  
}
```

- TypeScript only lets you use what **every** member of the union has
- **Narrowing** is proving which member you actually hold
- Inside the check, TS refines the type for you, no casting

typeof Narrows Primitives

```
function format(id: number | string): string {  
  if (typeof id === "string") {  
    return id.toUpperCase(); // narrowed to string  
  }  
  return id.toFixed(0);      // narrowed to number  
}
```

- Works for "string", "number", "boolean", "function", "object", "undefined", "bigint", "symbol"
- Remember `typeof null === "object"`; it will not narrow away `null`

`instanceof` Narrows Class Instances

```
function describe(x: unknown): string {  
  if (x instanceof Date) return `date: ${x.getFullYear()}`;   
  if (x instanceof Error) return `error: ${x.message}`;  
  return `other: ${String(x)}`;  
}
```

- The parameter is `unknown`, not `any`; TS refuses to touch it until you have proved something
- This is exactly why `unknown` is the safe choice for external data

in Narrows Object Shapes

```
type Circle = { radius: number };
type Square = { side: number };

function area(shape: Circle | Square): number {
  if ("radius" in shape) return Math.PI * shape.radius ** 2;
  return shape.side ** 2;
}
```

- Distinguishes shapes by which property exists
- Fine for two members; fragile once the union grows

Truthiness and Equality Narrowing

```
function greet(name?: string): string {  
  if (!name) return "Hello, stranger";  
  return `Hello, ${name.trim()}`; // definitely a string here  
}  
  
function check(x: string | null) {  
  if (x === null) return "empty";  
  return x.length; // null is gone  
}
```

- An early `return` narrows the **rest of the function**
- This is why guard clauses and TypeScript get along so well

Discriminated Unions

- Give every member a shared property whose type is a **literal**
- `switch` on that tag and each branch narrows to exactly one member

```
type Shape =  
  | { kind: "circle"; radius: number }  
  | { kind: "square"; side: number }  
  | { kind: "rect"; width: number; height: number };  
  
function area(s: Shape): number {  
  switch (s.kind) {  
    case "circle": return Math.PI * s.radius ** 2;  
    case "square": return s.side ** 2;  
    case "rect":   return s.width * s.height;  
  }  
}
```

Why the Tag Beats `in`

```
// With `in`, adding a member means auditing every check by hand
if ("radius" in shape) { /* ... */ }

// With a tag, the compiler tracks it for you
switch (shape.kind) { /* ... */ }
```

- One property to read, whatever the union's size
- Reliable even when two members share property names
- The standard way to model events, actions, and API results

Exhaustiveness With `never`

```
function assertNever(value: never): never {  
    throw new Error(`Unhandled: ${JSON.stringify(value)}`);  
}  
  
function area(s: Shape): number {  
    switch (s.kind) {  
        case "circle": return Math.PI * s.radius ** 2;  
        case "square": return s.side ** 2;  
        case "rect":    return s.width * s.height;  
        default:        return assertNever(s); // s is `never` here  
    }  
}
```

- Add a fourth member and forget its `case` → **this line stops compiling**
- The compiler finds the missing branch for you, before runtime

User-Defined Type Guards

- A function whose return type is `x is Type` narrows at the **call site**

```
type Cat = { kind: "cat"; meow(): void };
type Dog = { kind: "dog"; bark(): void };

function isCat(pet: Cat | Dog): pet is Cat {
    return pet.kind === "cat";
}

const pets: (Cat | Dog)[] = loadPets();

const cats = pets.filter(isCat); // Cat[] - narrowed!
cats[0].meow();                 // safe
```

- A plain `boolean` return would leave `cats` as `(Cat | Dog)[]`

Switching on the Tag

Narrowing and Discriminated Unions

Give every member a shared tag, switch on it, and TypeScript knows exactly which shape you are holding.

```
type Shape =  
  | { kind: 'circle'; radius: number }  
  | { kind: 'square'; side: number }  
  | { kind: 'rect'; w: number; h: number };
```

kind is the DISCRIMINANT: a literal type unique to each member.

switch (s.kind)

case 'circle'
s is narrowed to the circle member.
s.radius is available; s.side is an error.

Why a tag beats a guess

Without a discriminant you end up probing for properties ('radius' in s), which breaks the moment two members share a field. A literal tag is unambiguous, cheap at run time, and survives JSON, so the same union works for data off the network. Narrowing is flow analysis: TypeScript tracks what must be true on each path. Reassigning the variable throws that away.

case 'square'
s.side is available, and nothing else.

case 'rect'
s.w and s.h are available.

default
const check: never = s;
All three handled, so s is never here; it compiles.
Add a fourth member and THIS LINE fails to compile, which is how you are told to update the switch.

Everything that narrows

typeof x === 'string'
for primitives
x instanceof Date
for class instances
'radius' in s
for plain object shapes
Array.isArray(x)
for arrays
if (x) / if (x !== null)
truthiness and null
x.kind === 'circle'
the discriminant: the one that scales

- Each branch narrows the union to exactly one member
- `never` in the default proves you covered every case